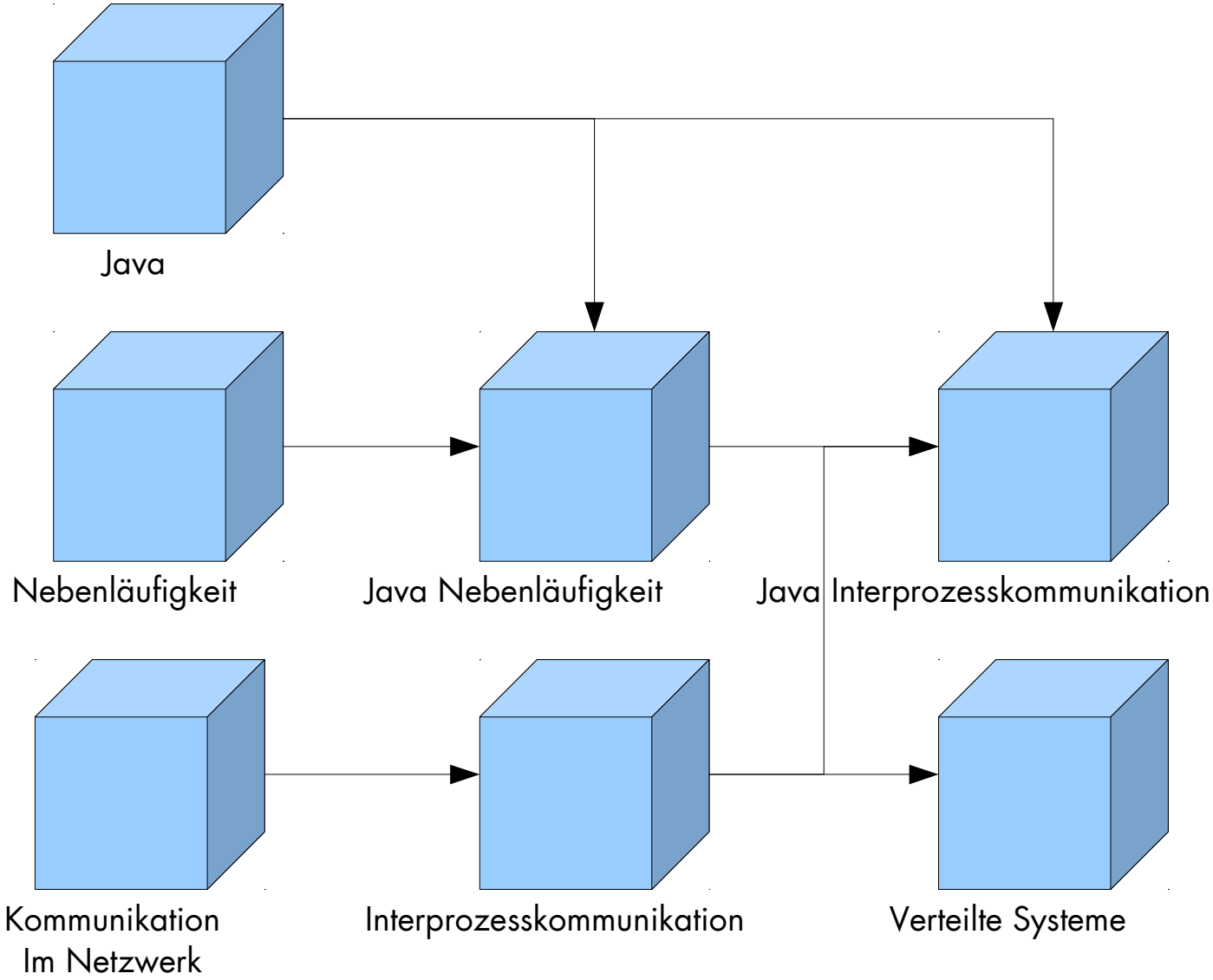
UNIVERSITÄT LEIPZIG
Institut für Wirtschaftsinformatik

Entwicklung verteilter Anwendungen

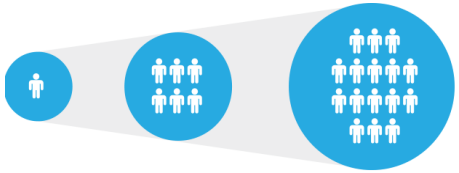
Middleware

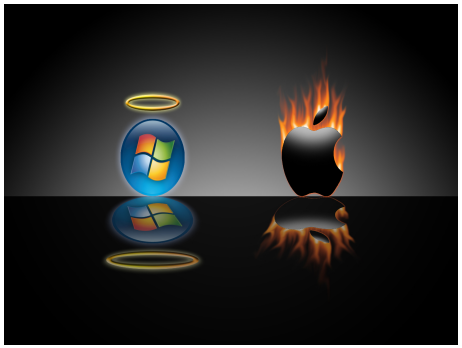
Pascal Kovacs

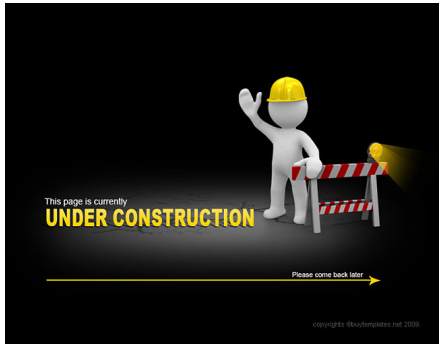
4. Juni 2020

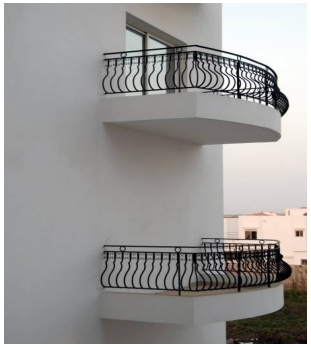

Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation
Im Netzwerk
Interprozesskommunikation
Verteilte Systeme

Nicht funktionale Anforderungen (NFA)

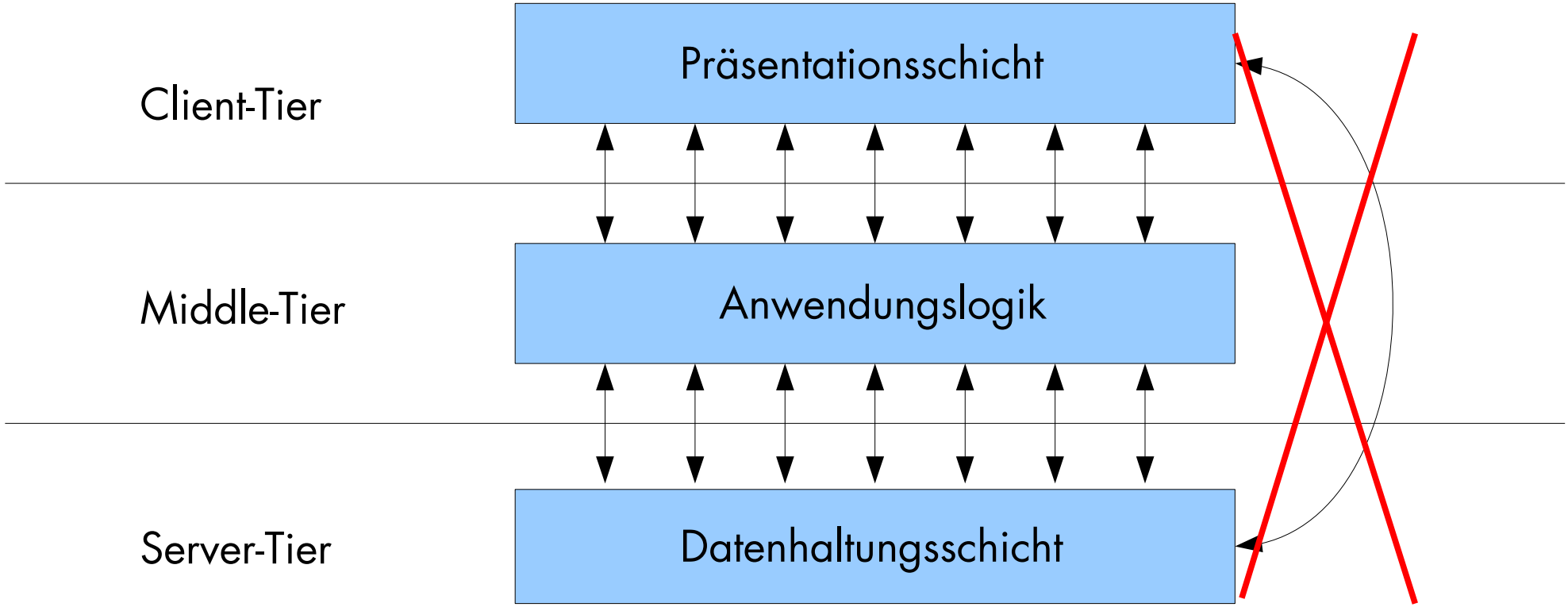




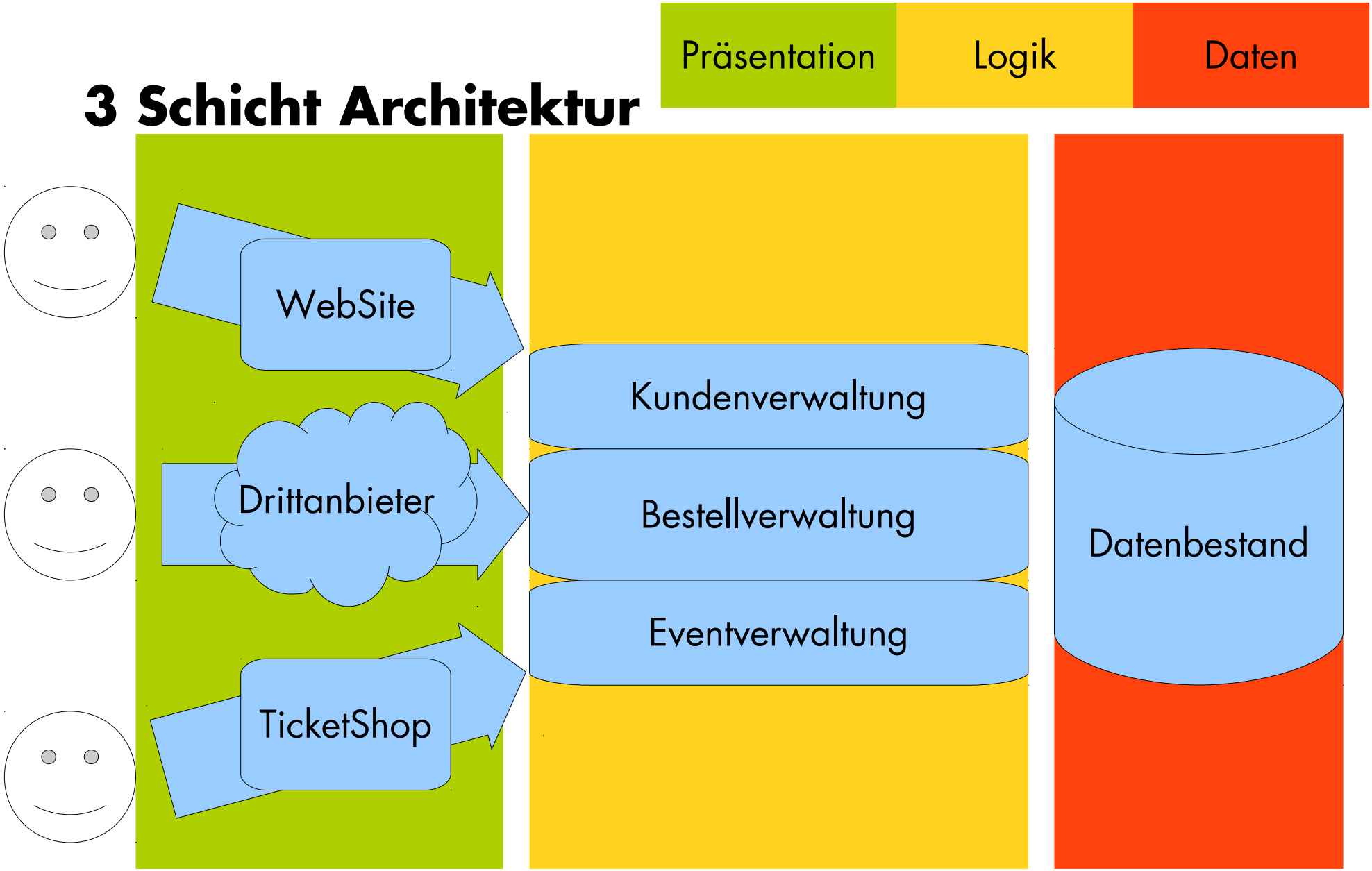




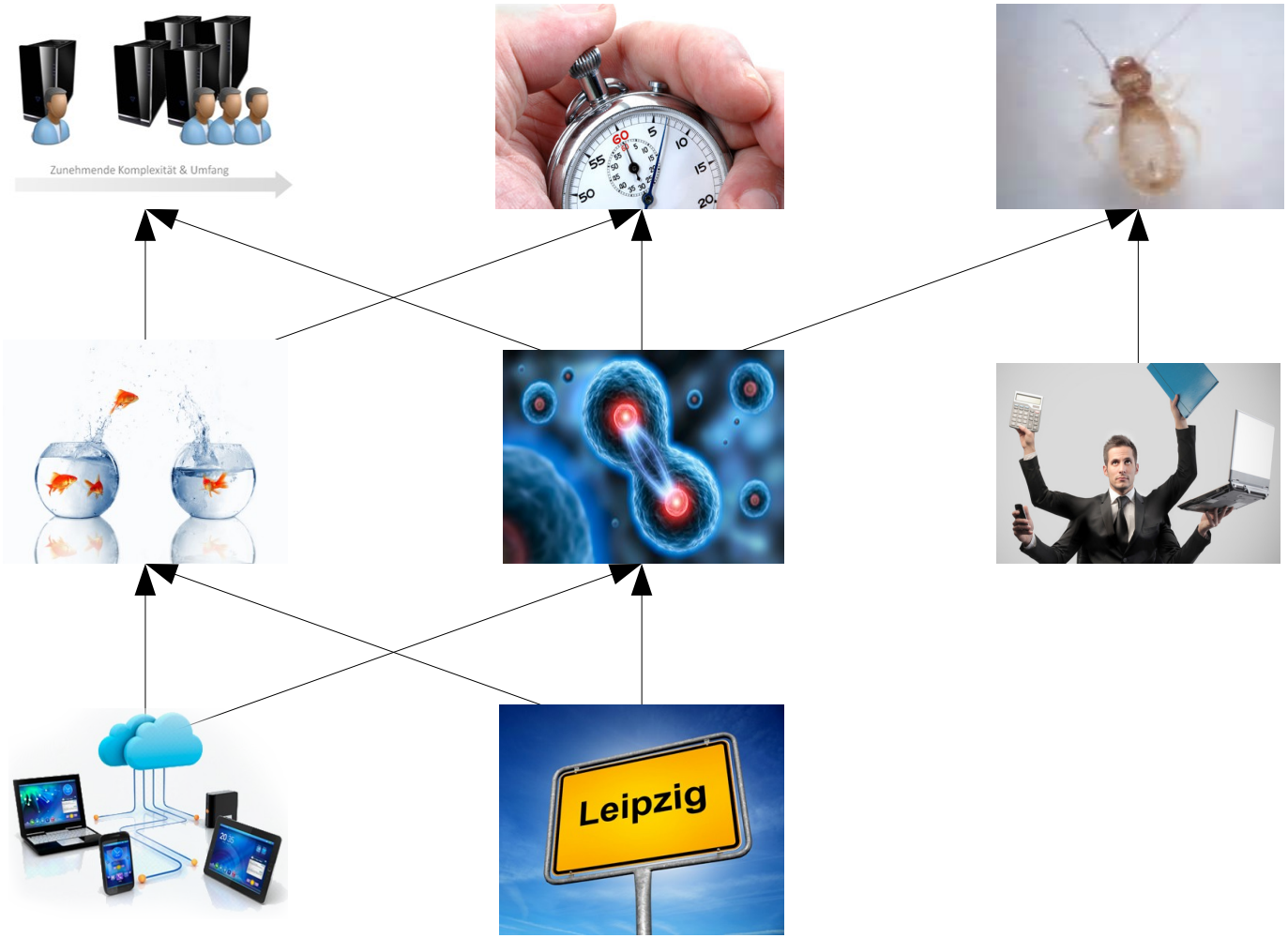
3-Tier-Architektur



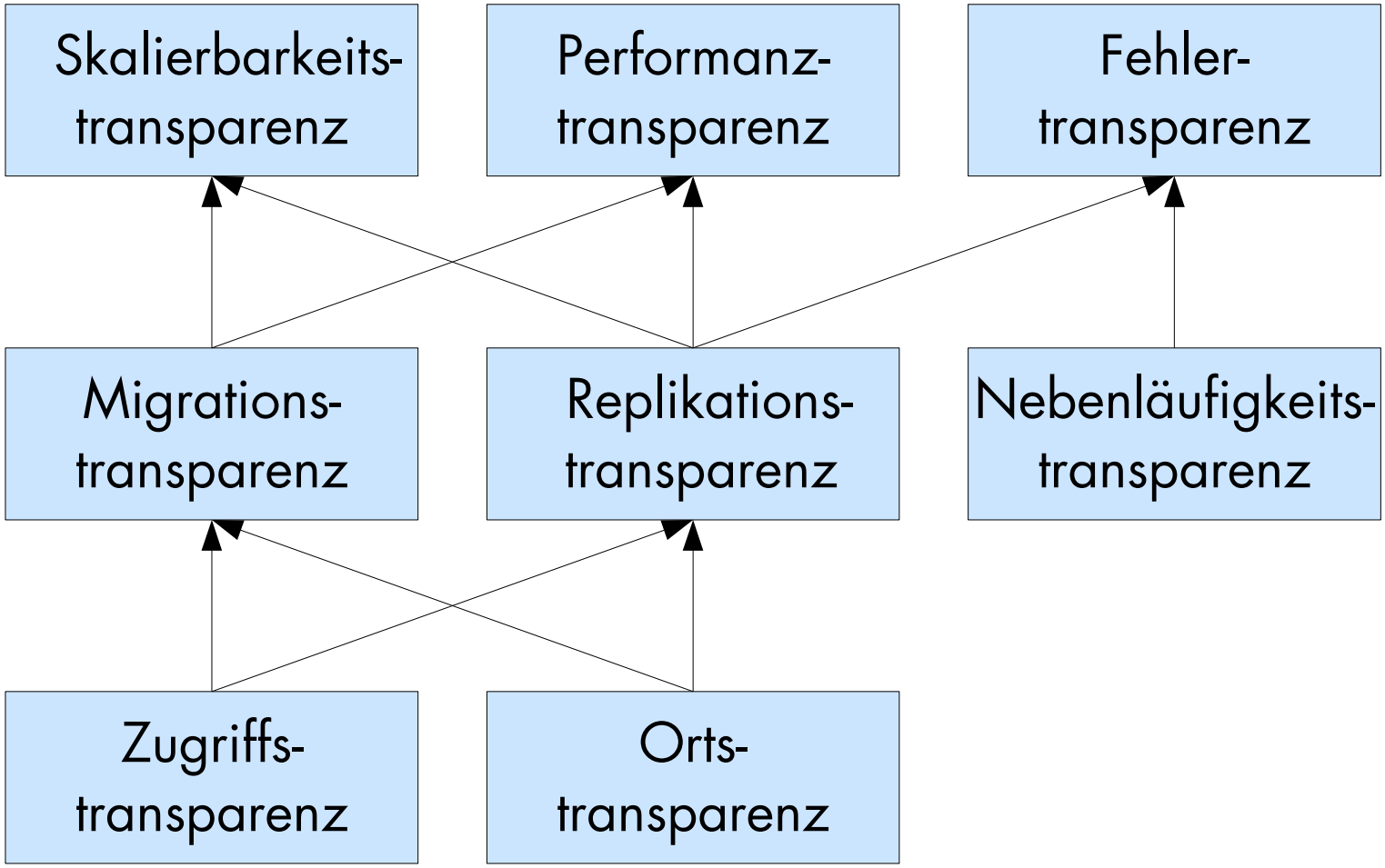
3 Schicht Architektur



Dimensionen der Transparenz

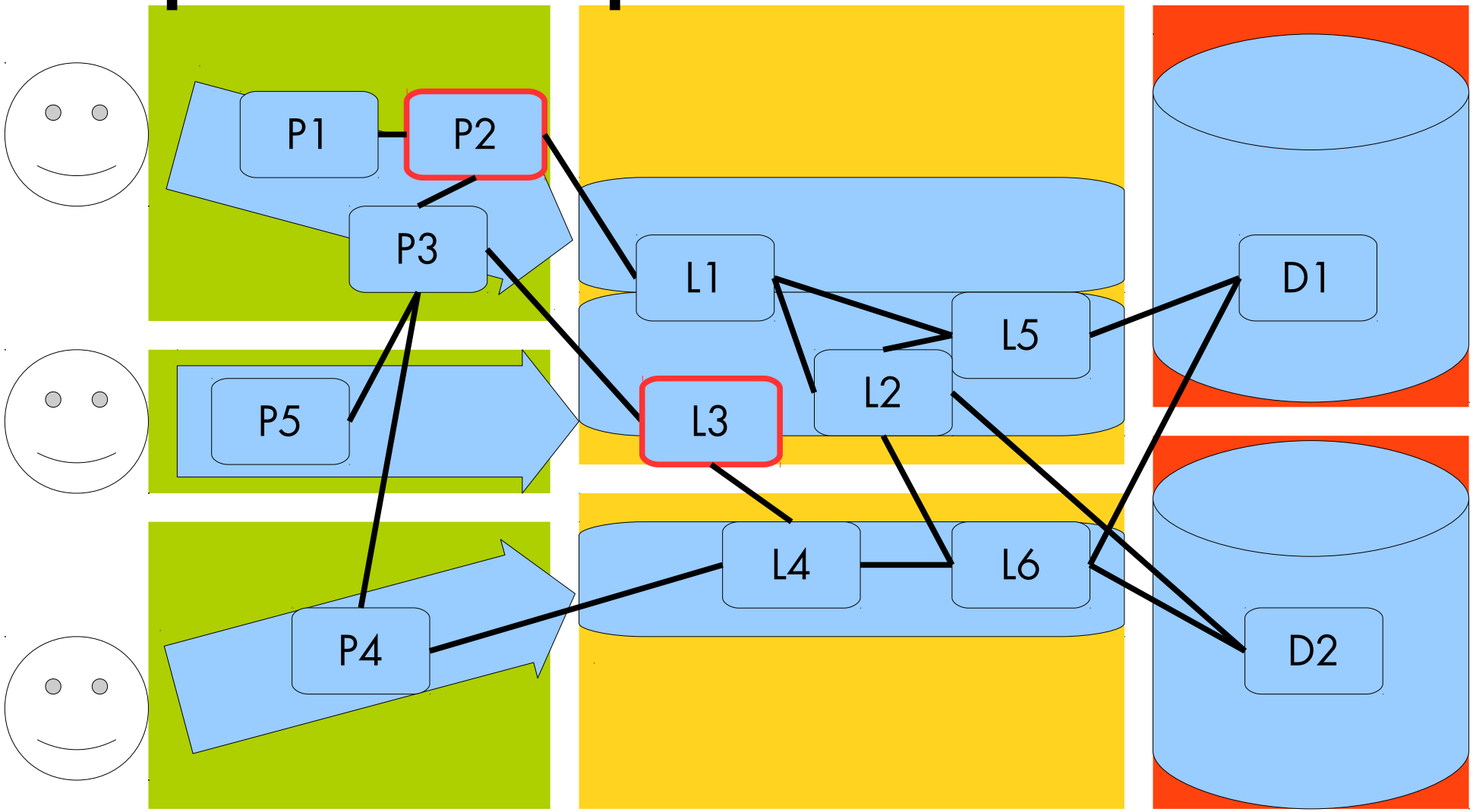


Dimensionen der Transparenz

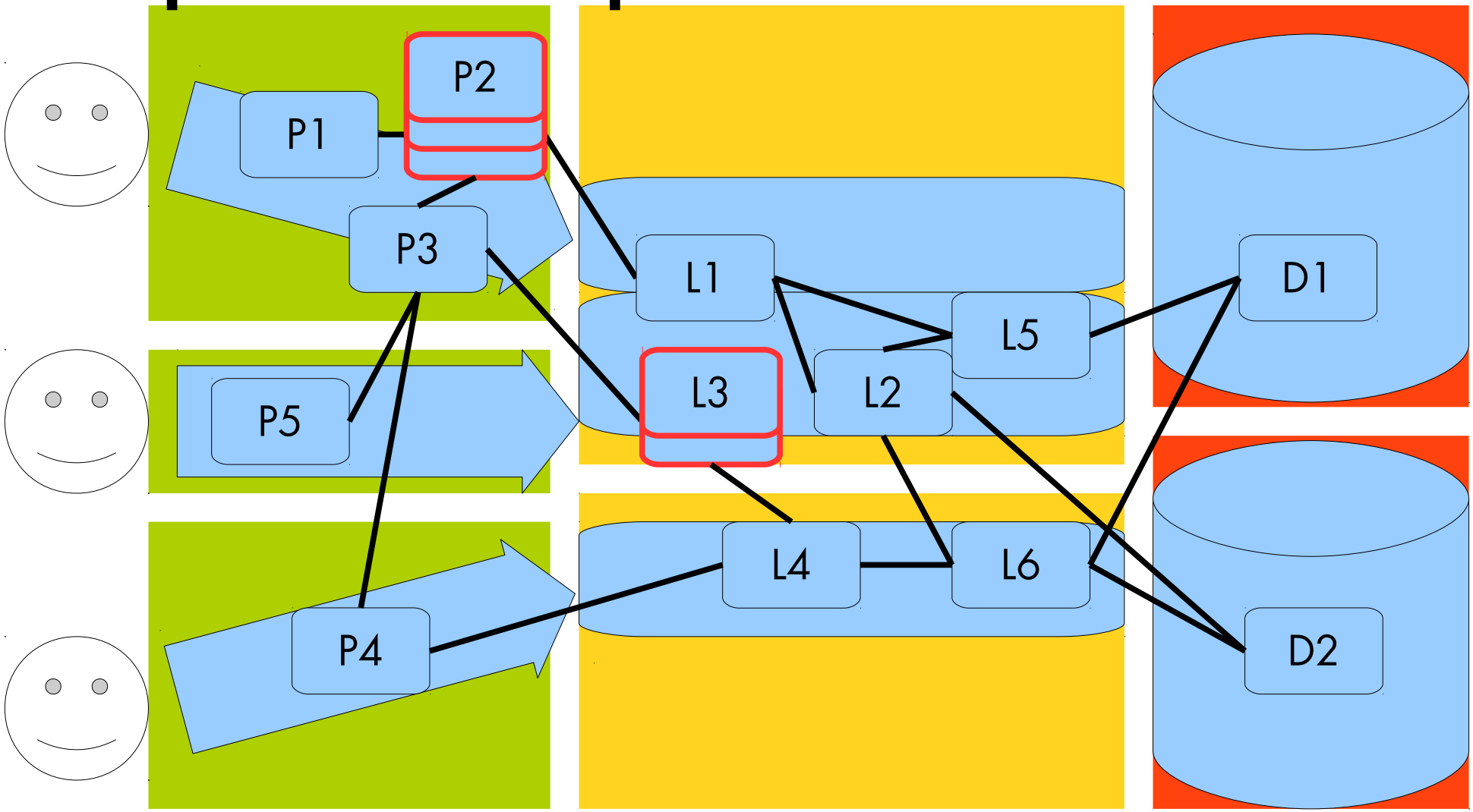


Quelle: Emmerich, W.: Konstruktion von verteilten Objekten, S. 27

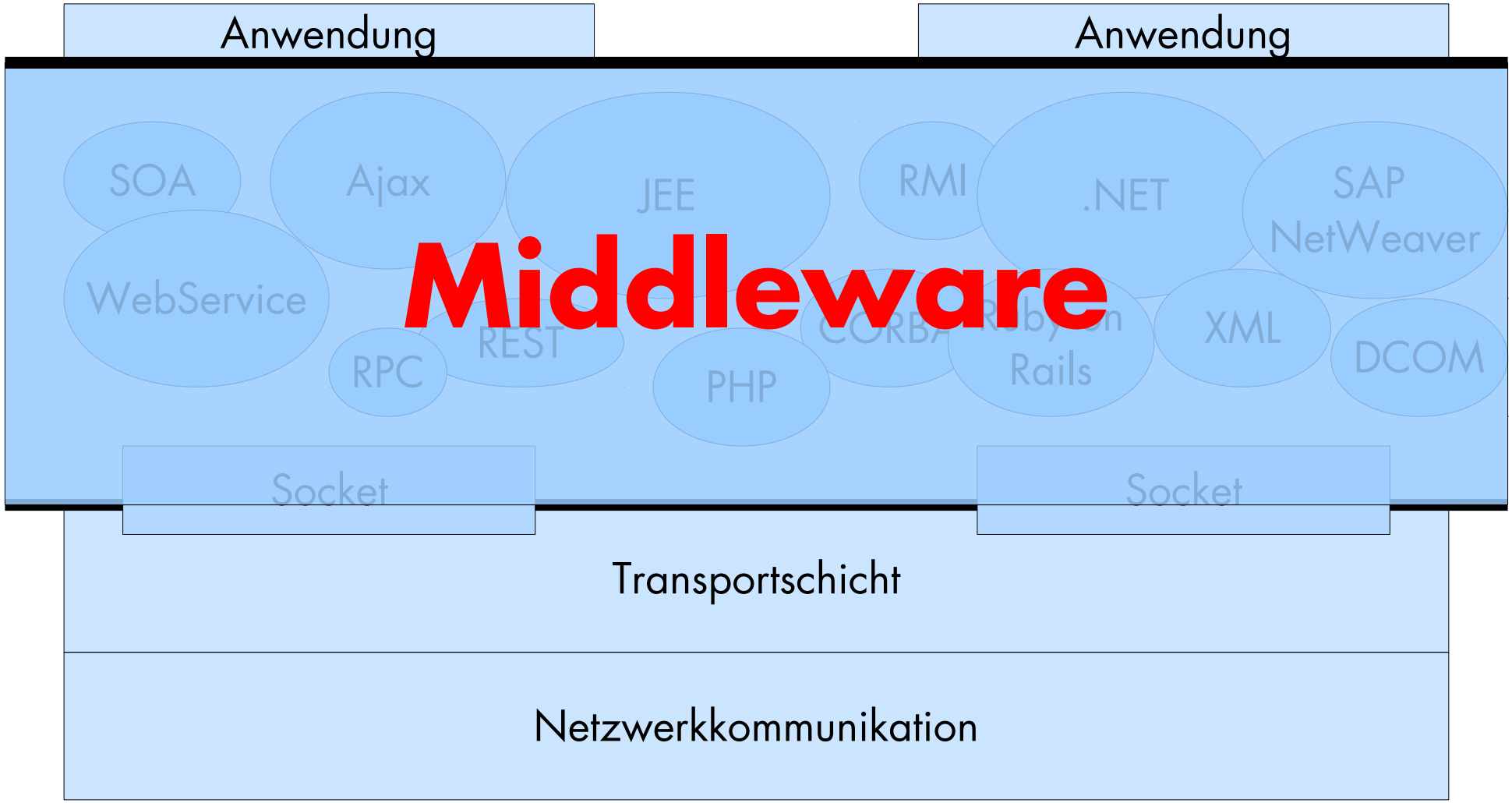
Replikationstransparenz



Replikationstransparenz



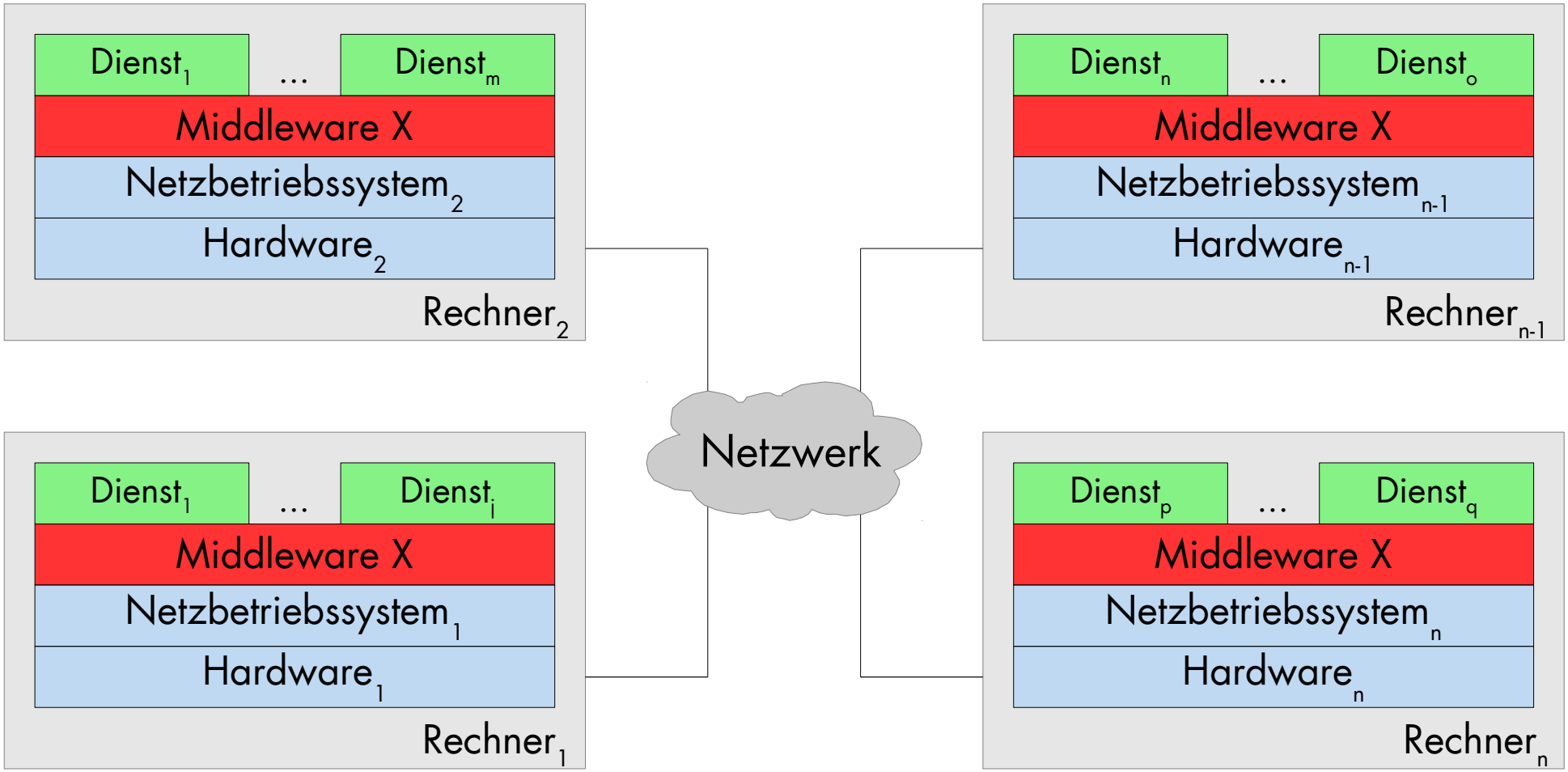
Middleware


Anwendung
Anwendung
SOA
Ajax
JEE
RMI
.NET
SAP NetWeaver
WebService
Middleware
RPC
REST
PHP
CORBA
Ruby on Rails
XML
DCOM
Socket
Socket
Transportschicht
Netzwerkkommunikation

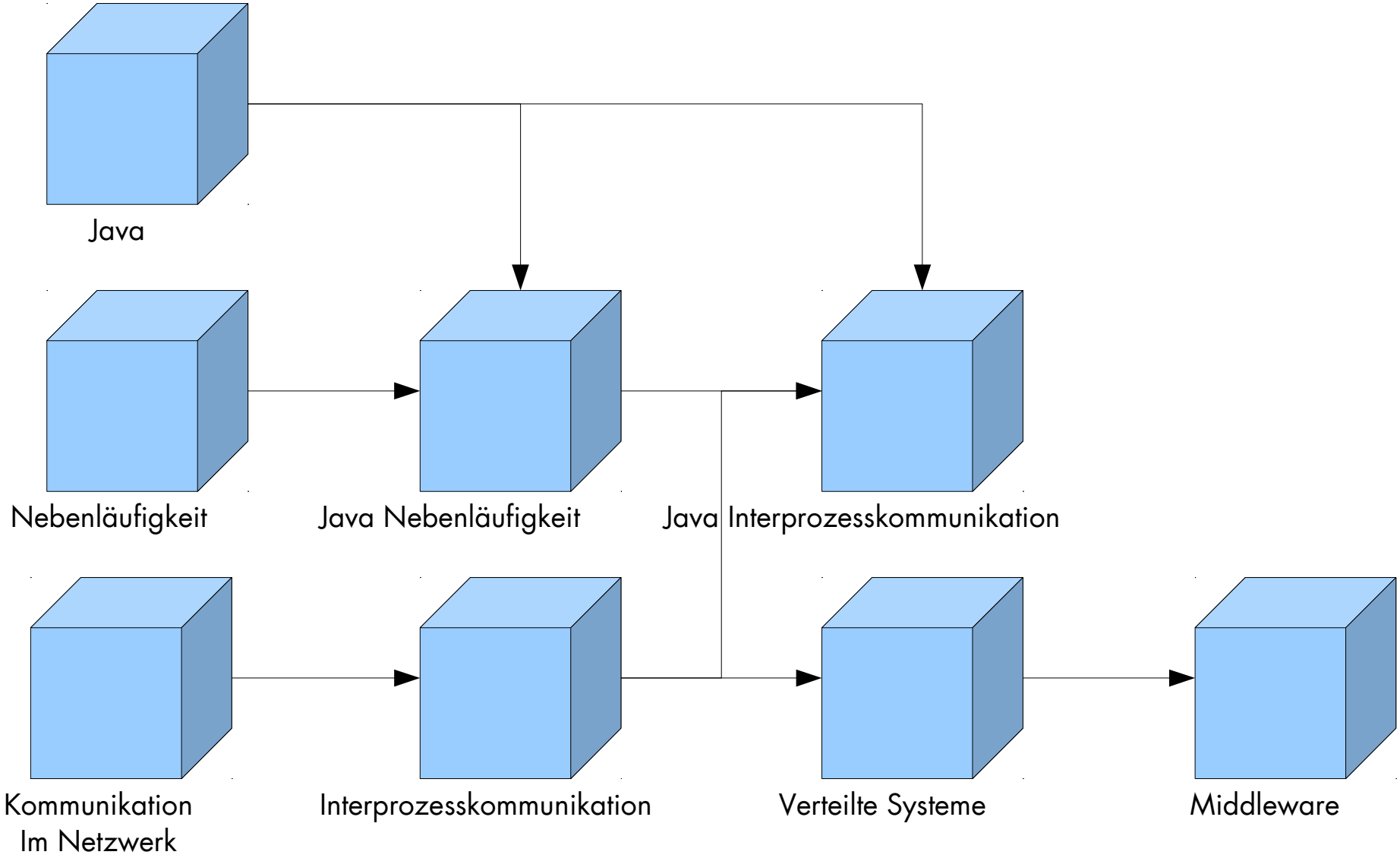
Motivation für Middleware

- NFA und Transparenz bedingen **verteilte** komplexe Kommunikation
- Handling der Kommunikation sehr Aufwändig und Fehleranfällig
- Zusätzliche Herausforderung durch Heterogenität der Systeme
  - Hardware
  - Betriebssystem
  - Programmiersprache
  - Versionen
- Middleware als einheitliche Plattform für Entwickler verteilter Systeme
  - Auslagerung technischer Details der Kommunikation
  - Konzentration auf Realisierung der fachlichen Anwendung
  - Verfügbarkeit ergänzender Dienste(Naming, Security und Transaction)
- Idealvorstellung: Baukasten lose gekoppelter interoperabler Dienste

Modell eines verteilten Systems



Ähnlich zu: Emmerich, W.: Konstruktion von verteilten Objekten, S. 9


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation
Im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

Ziele Middleware

- Mehrwert kommunikationsorientierter Middleware kennen
- 3 Modelle der kommunikationsorientierter Middleware kennen
- Mehrwert anwendungsorientierter Middleware verstehen
- Dienste von anwendungsorientierter Middleware kennen
- Komponentenparadigma verstehen

Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

Ziele kommunikationsorienter Middleware

- Abstraktion von der Netzwerkprogrammierung (Sockets)
- Prozeduren/Funktionen/Methoden – Aufruf statt Strömen
- Unterstützung Datenübertragung zwischen entfernten Komponenten
- Erweiterung der Netzverbindung durch Dienste
  - Datenanpassungen: Marshalling/Unmarshalling, Formatekonvertierung
  - Fehlerbehandlung : Prüfsummen, Ausfallbehandlung, Sendewiederholung
- Realisierung durch Middlewareprotokoll
- Setzt auf Protokollen IP, TCP/UDP auf

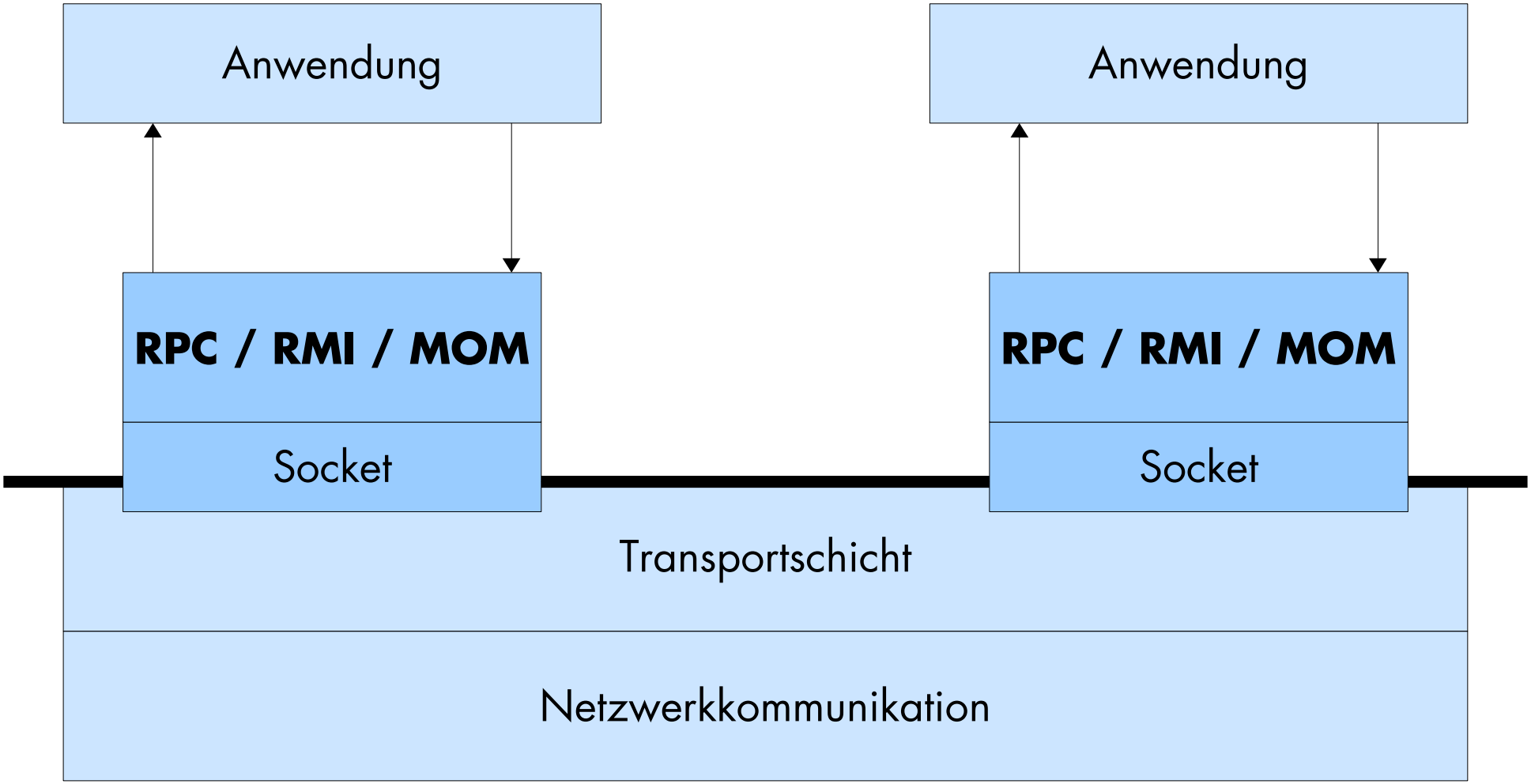
Datentransformation & Heterogenität

- Datentransformation
  - (Un)Marshalling der Aufrufparameter in übertragungsfähiges Format
  - Sicherung eines einheitlich definierten Datentyps für Sender&Empfänger
- Heterogenität Hardware und Betriebssystem
  - Ausgleichen der Heterogenität verschiedener OS/HW Kombinationen
  - Zulassen Individualität der einzelnen Knoten (z.B. Endian-Darstellung)
  - Übertragung in vereinbarter Network Byte Order
  - Unterschiedliche Formate der Zeichendarstellung in Betriebssystemen (IBM-Mainframes: EBCDIC-Format, Windows, UNIX – ASCII-Code)

Programmiermodelle

- 1) Prozedurales Modell
  - Entfernter Prozeduraufruf (Remote Procedure Call RPC)
  - Synchrone Kommunikation
- 2) Objektorientiertes Modell
  - Entfernter Methodenaufruf (Remote Method Invocation RMI)
  - Synchrone Kommunikation
- 3) Nachrichtenbasiertes Modell
  - Nachrichtenversand (Message oriented Middleware MOM)
  - Verschicken/Empfangen von Nachrichten
  - Asynchrone Kommunikation über Warteschlangen

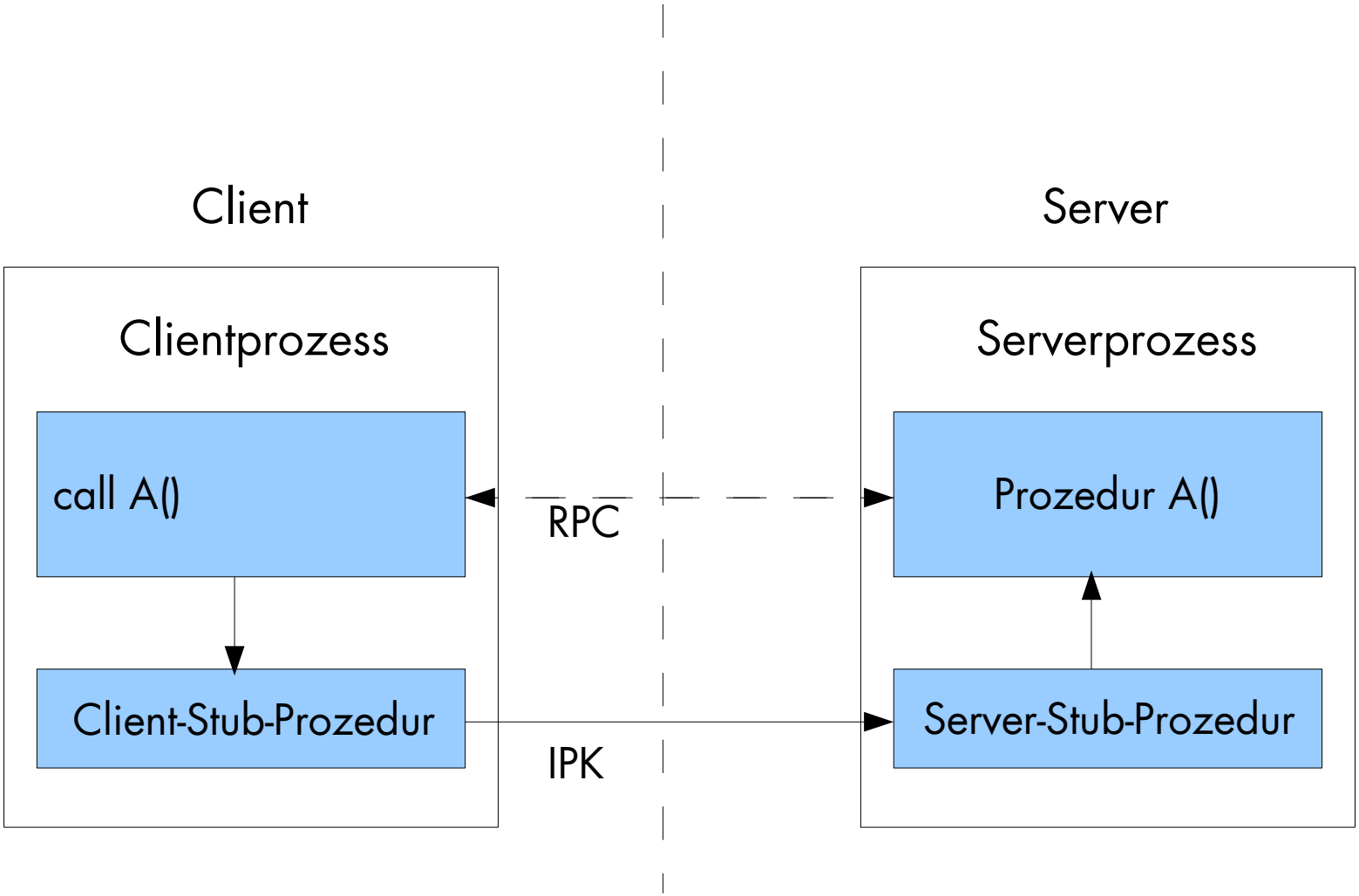
RFC/RMI/MOM und Sockets



Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

1) Remote Procedure Call (RPC)



Backup

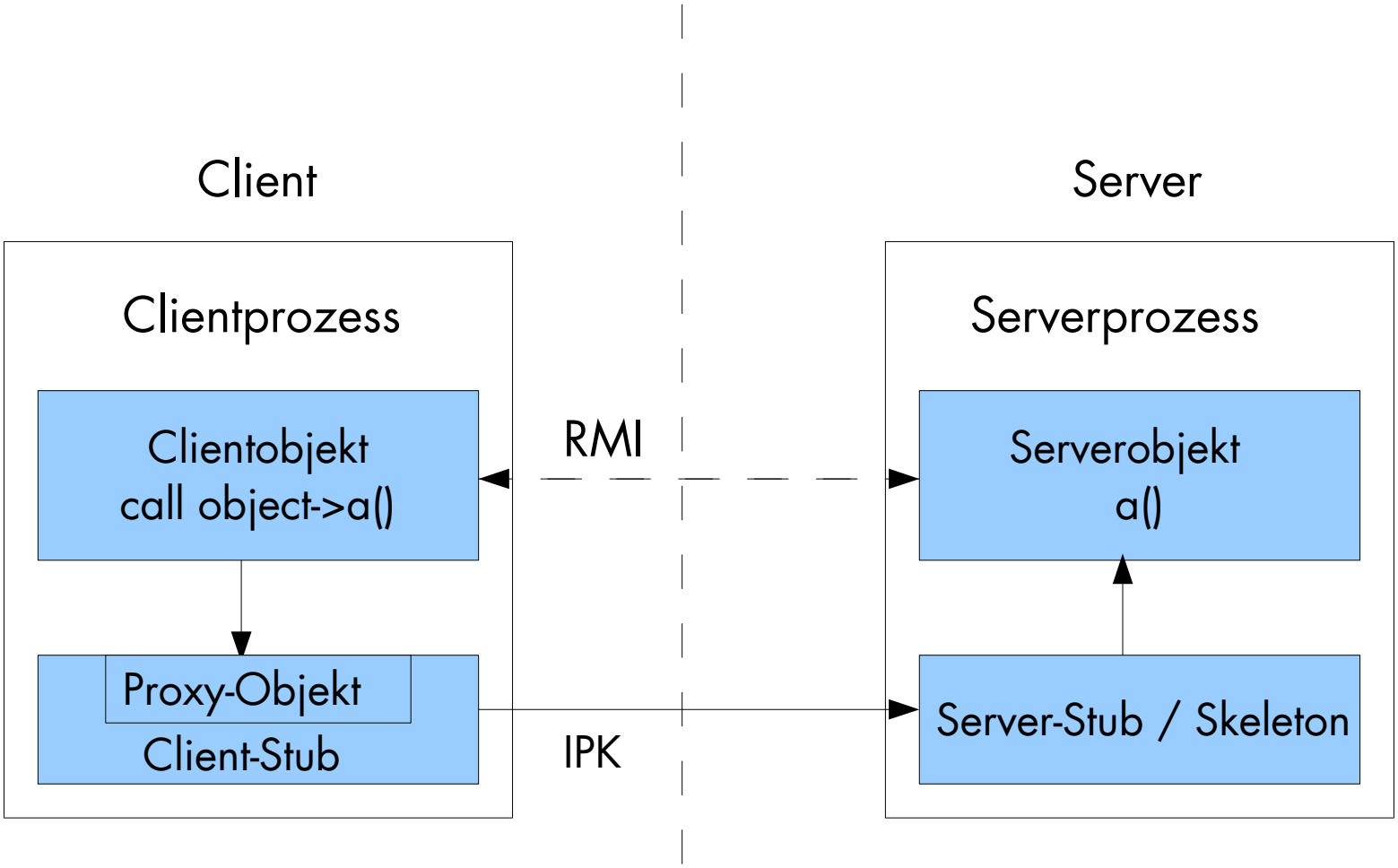
Aufruf von verteilten Prozeduren (RPC)

- Sockets sind niedrigste Form der Interprozesskommunikation(IPK)
- Interpretation der Nachrichten muss von Hand implementiert werden
- Entwicklung komplexerer Funktionen aufwändig und fehleranfällig
- IPK besser durch expliziten Aufruf von entfernten Prozeduren
- Konzept des Remote Procedure Call (RPC)
- Beruht auf lokalen Stellvertretern(Stubs)
- Simulieren normalen synchronen Prozeduraufruf
- Leiten den Aufruf über Sockets an Server weiter
- Server nimmt dies ebenfalls als normalen Prozeduraufruf war
- Somit zuverlässige Syntax- und Aufrufbehandlung
- Ggf. Typkonvertierungen und Formatanpassungen

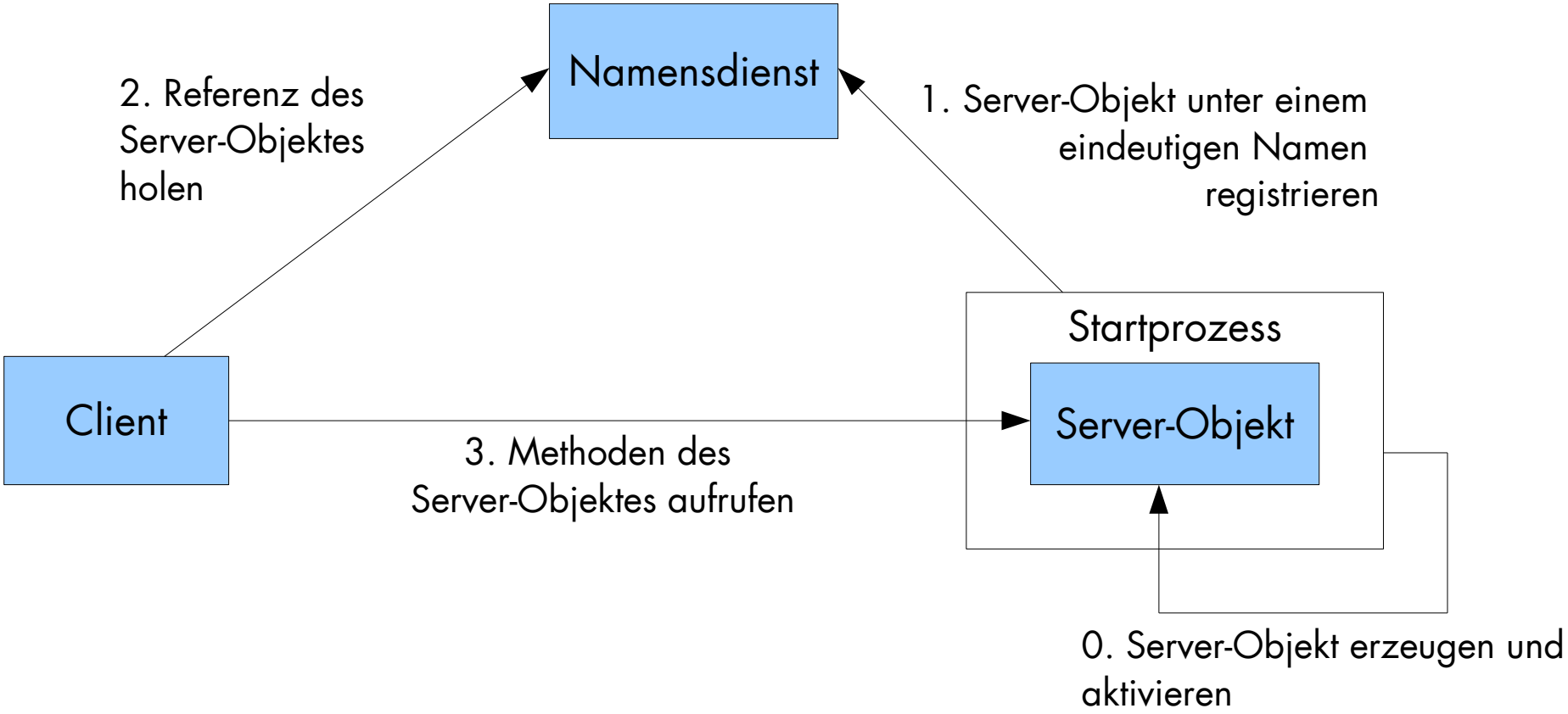
Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

Remote Method Invocation (RMI)



Aufbau RMI


Namensdienst
2. Referenz des Server-Objektes holen
1. Server-Objekt unter einem eindeutigen Namen registrieren
Startprozess
Client
Server-Objekt
3. Methoden des Server-Objektes aufrufen
0. Server-Objekt erzeugen und aktivieren

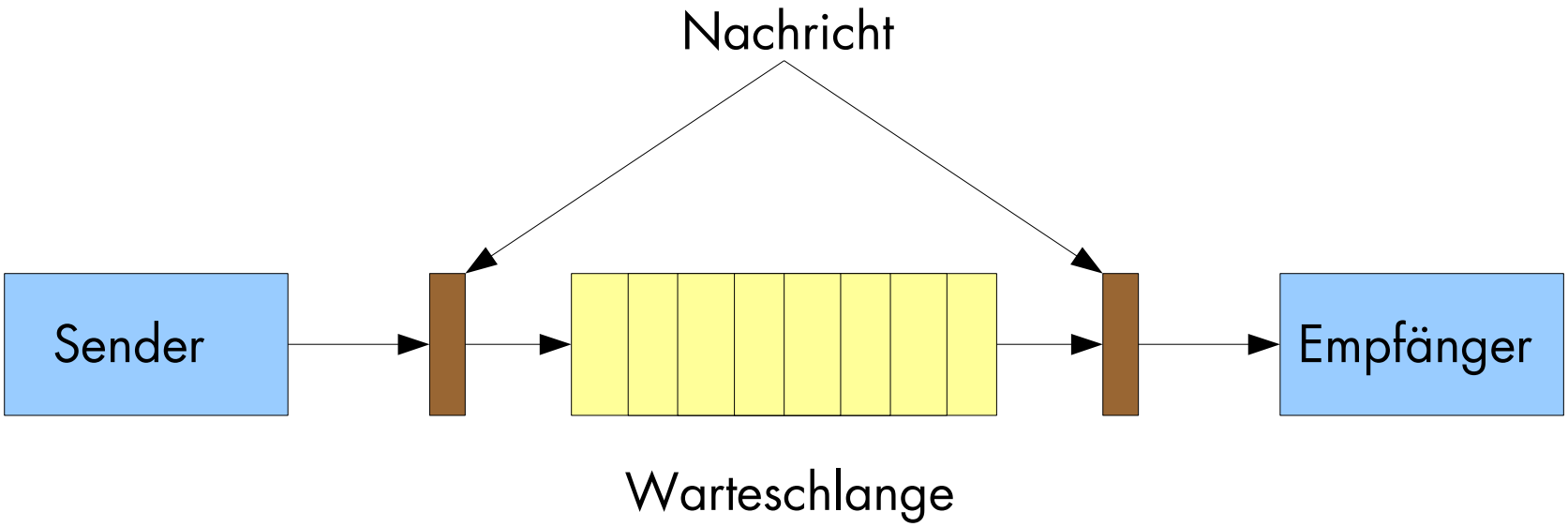
Remote Method Invocation (RMI)

- Objektorientierte Variante des *Remote Procedure Call* (RPC) in Java
- Konzept auf verteilten Objekten mit aufrufbaren Methoden
- Synchrone Kommunikation mit Aufruf und Ergebnisrückgabe
- Nutzt ebenfalls Stubs für transparente Kommunikation
- Zusätzlich auf Clientseite ein Stub-Objekt genannt Proxy
- Simuliert das entfernte Objekte
- Server-Stub wird auch als Skeleton bezeichnet

Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware

Message oriented Middleware (MOM) Beispiel Logging und Performanzeinfluss



Architektur

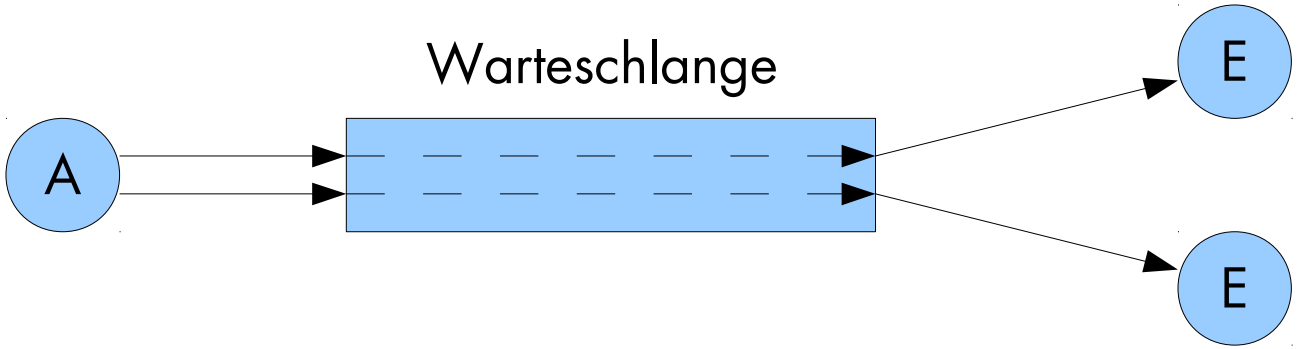
- Bestandteile sind Nachrichten (Messages), Warteschlangen (Queues), Warteschlangen-Verwalter (Manager)
- Warteschlangen-Verwalter organisiert mehrere Queues für unterschiedliche Empfänger
  - Zuordnung von Nachrichten zu Warteschlangen
  - Benachrichtigung der Empfänger über Eintreffen der Nachricht
  - Initialisierung und Überwachung der Warteschlange
  - Zugriffsschnittstelle und Übertragungsprotokoll
- Keine garantierte Auslieferung (falls garantierte Auslieferung gefordert – Konzept der persistenten Warteschlange)

Nachrichtenbasiertes Modell

- Messaging – asynchrones Programmiermodell auf Nachrichten
  - Nachrichtenformat von Middleware-Technologie vorgegeben
  - Verpackung von Daten und Aufrufinformationen
  - Werden direkt an Empfänger verschickt
  - Keine Garantie des Übertragungserfolgs/Verarbeitung am Ziel
  - Keine direkte Antwort auf Nachricht (asynchron)
- Vorteile
  - Sender und Empfänger weitgehend voneinander unabhängig
  - Empfänger bestimmt Zeitpunkt der Nachrichtenentgegennahme selbst
- Nachteile
  - Sender weiß nicht, ob Empfänger empfangsbereit
  - Keine Garantie hinsichtlich Verarbeitung

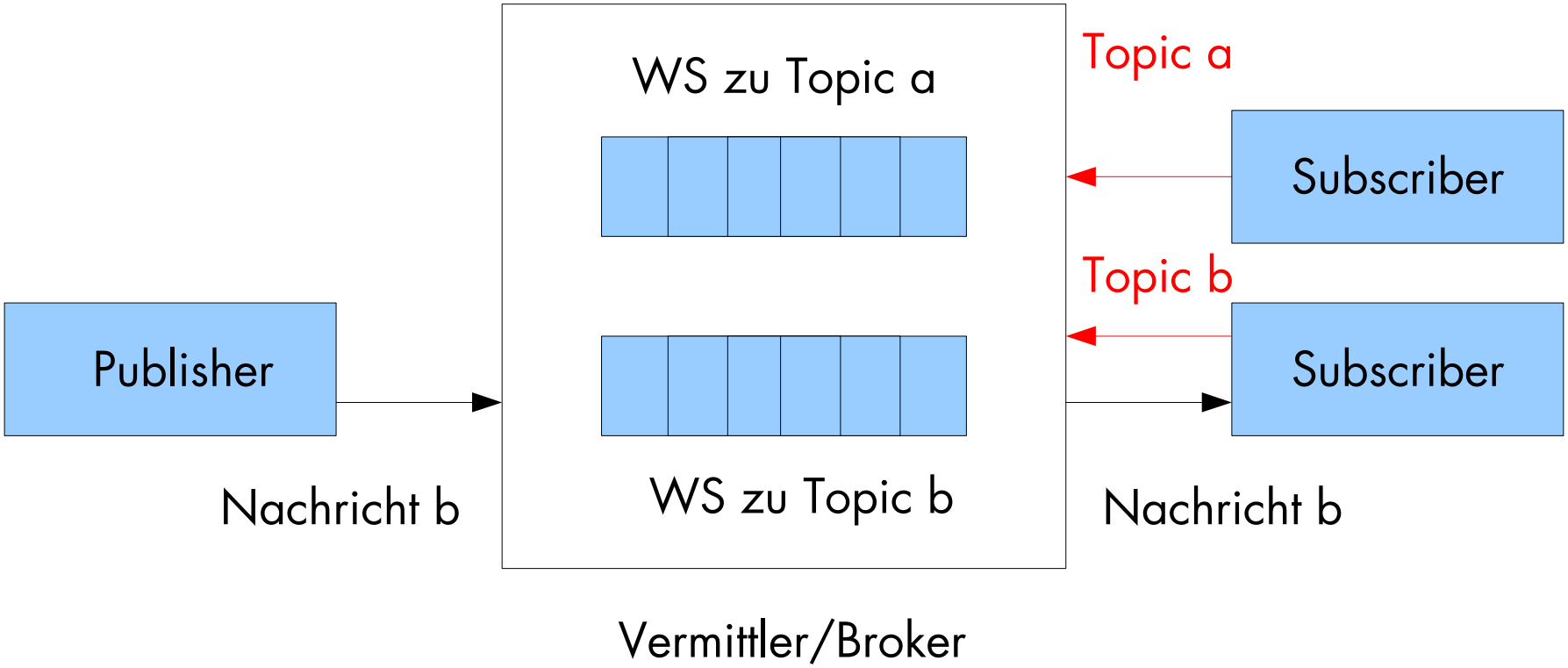
1) Point-to-Point-Modell (Standard)

- Absender stellen Nachrichten in Warteschlange ein
- Von dort Auslieferung an Empfänger
- Jede Nachricht nur an einen Empfänger ausgeliefert
- Nachrichten können auf mehrere Empfänger verteilt werden



2) Publish-Subscribe-Modell

- Simuliert ein Abonnentensystem

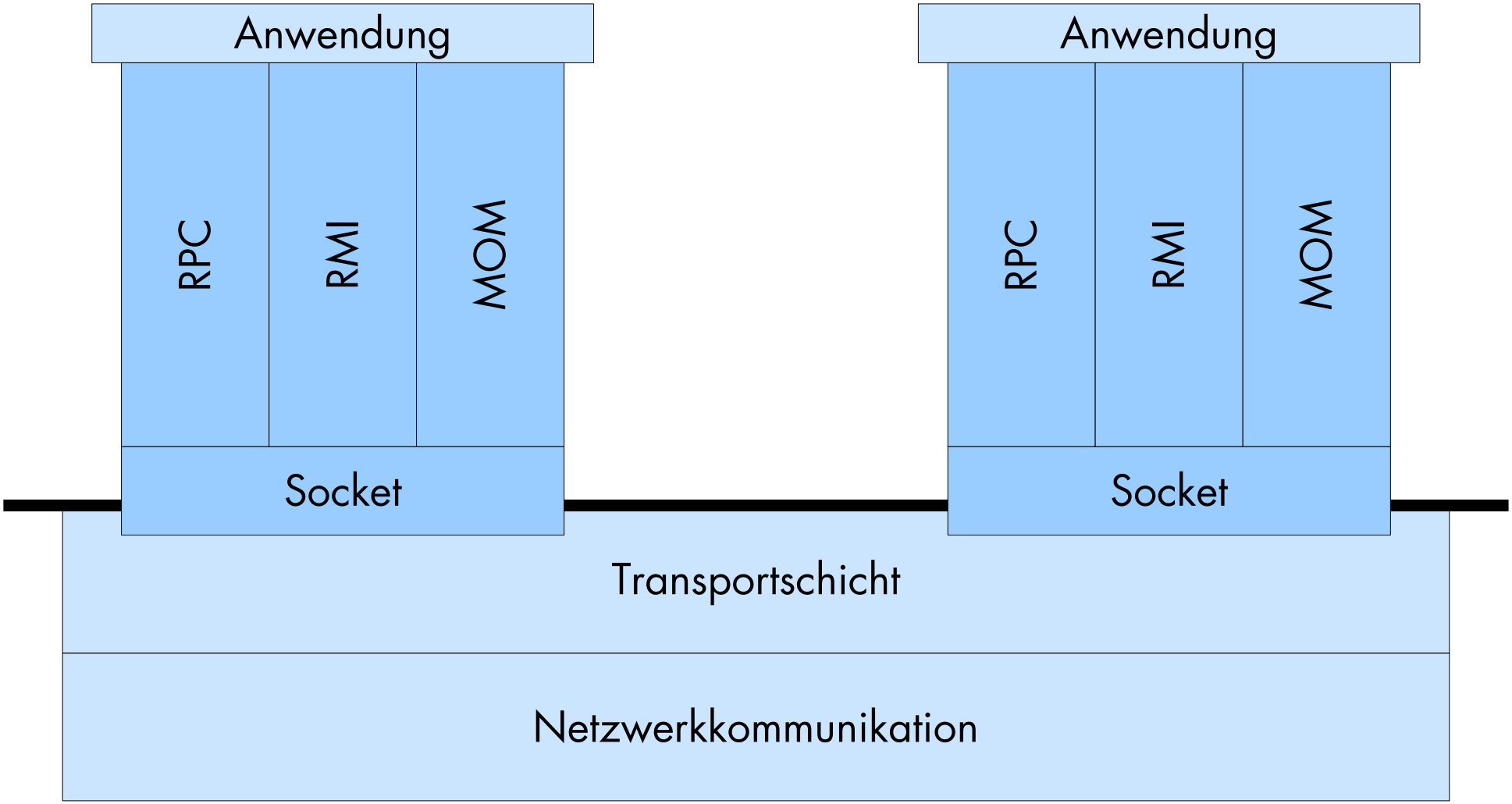


Backup

2) Publish-Subscribe-Modell

- Sender(Publisher) veröffentlichen Nachrichten
- Empfänger(Subscriber) abonnieren Nachrichten zu Thema
- Broker übernimmt Verteilung der Nachricht
- Publisher und Subscriber vollständig unabhängig (kennen sich nicht)
- Anwendungsbeispiele: Börsenticker, RSS-Fedds

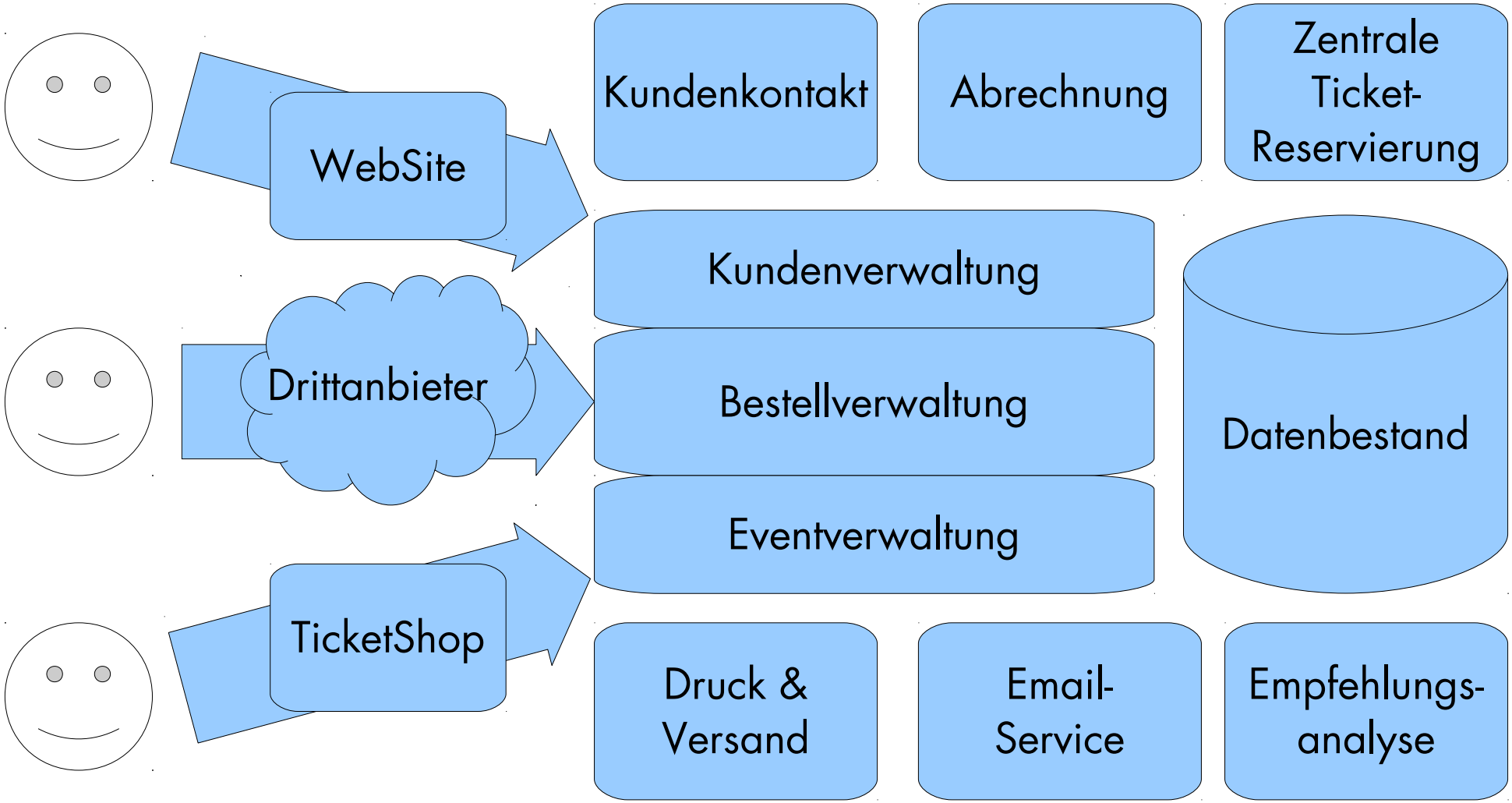
Kommunikationsorientierte Middleware



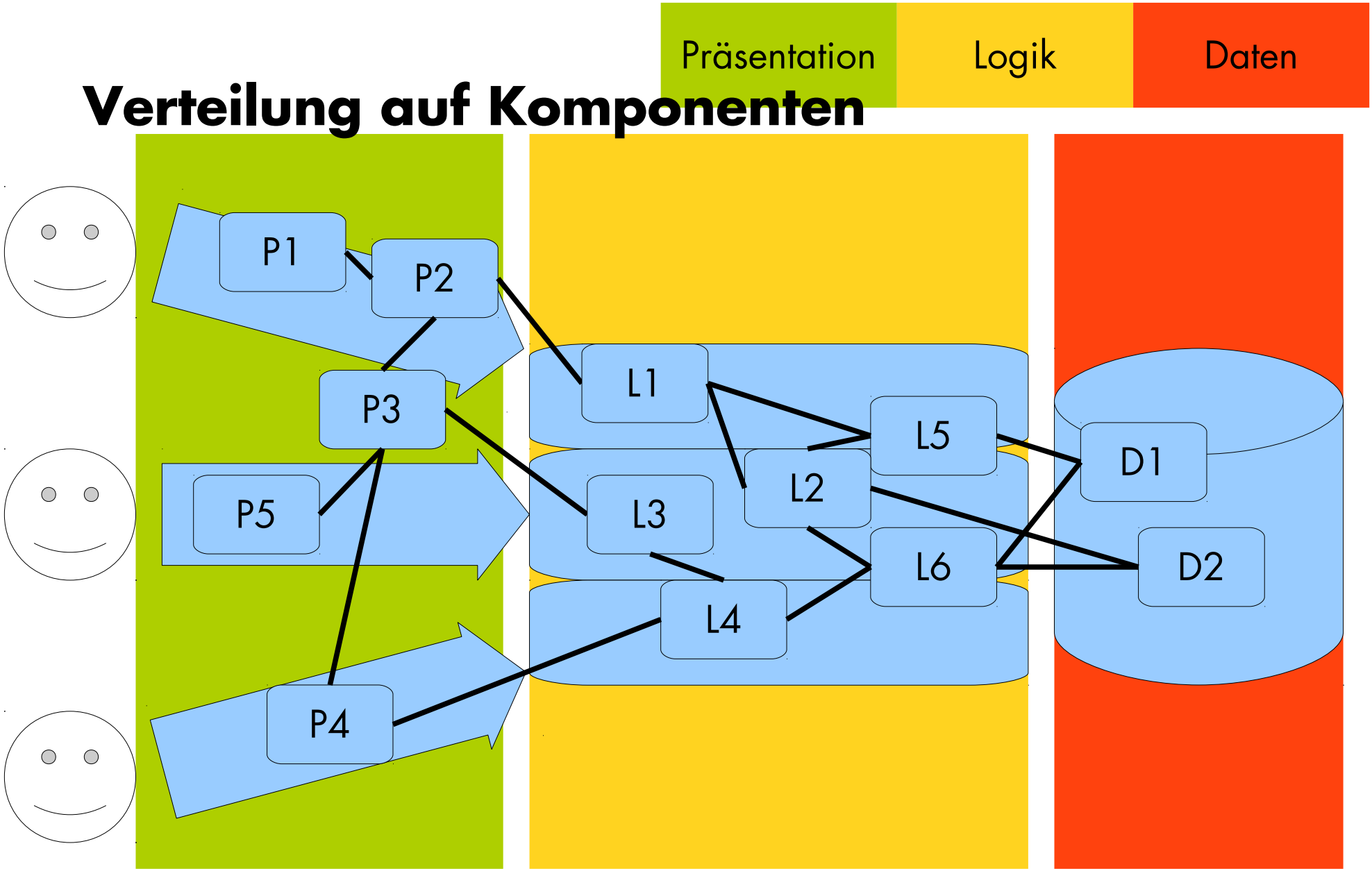
Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

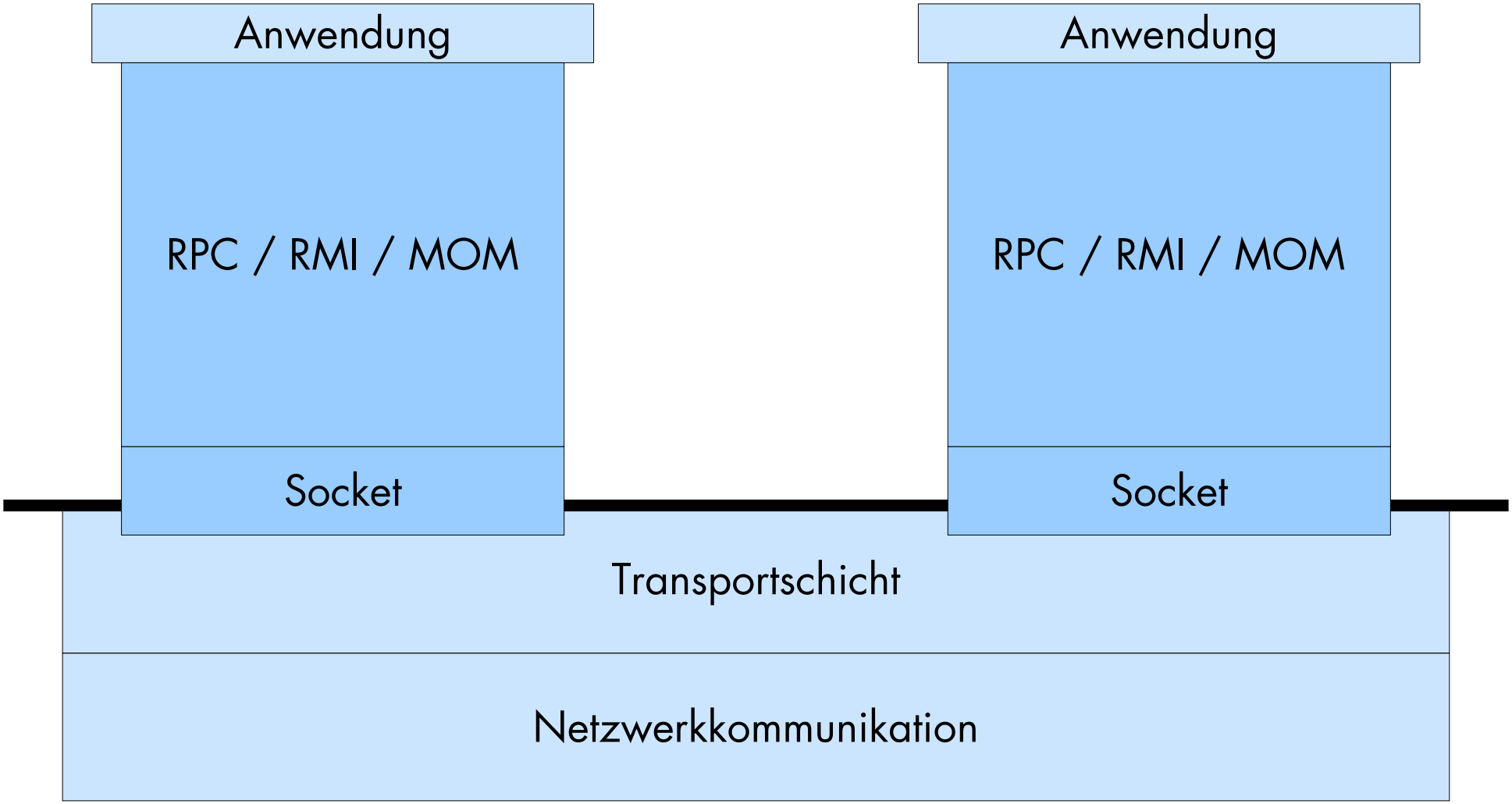
Übersicht Ticketshop



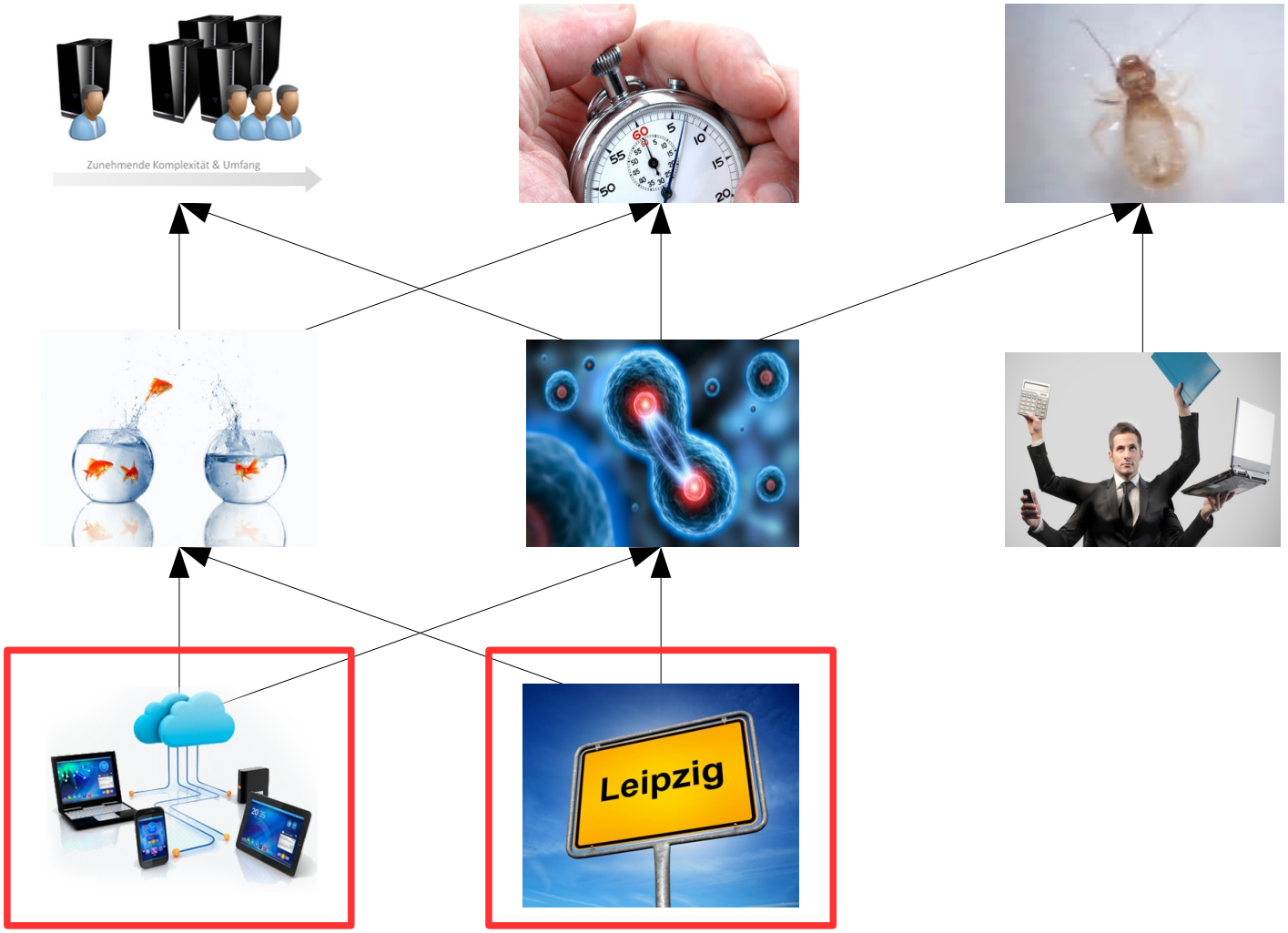
Verteilung auf Komponenten



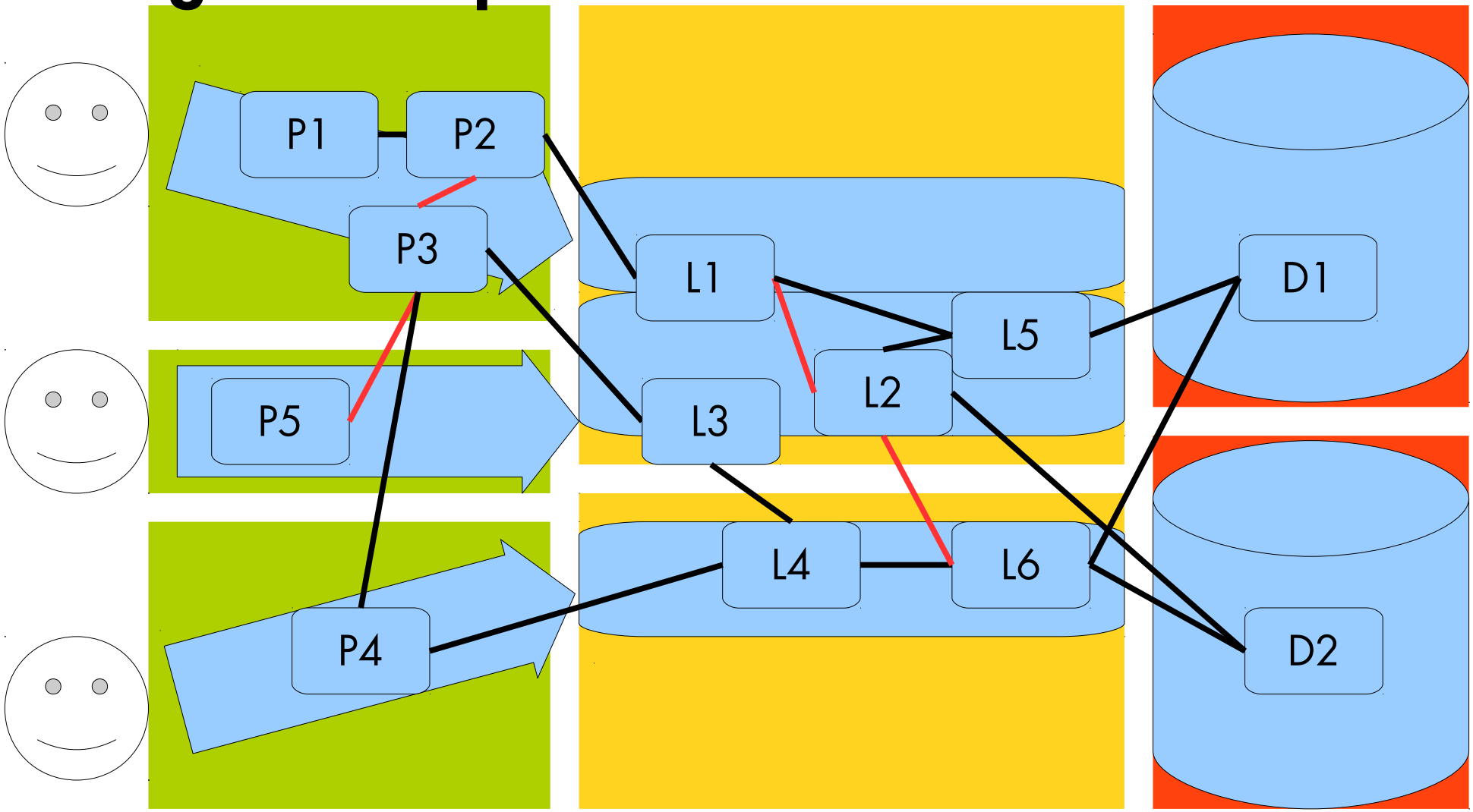
Kommunikationsorientierte Middleware



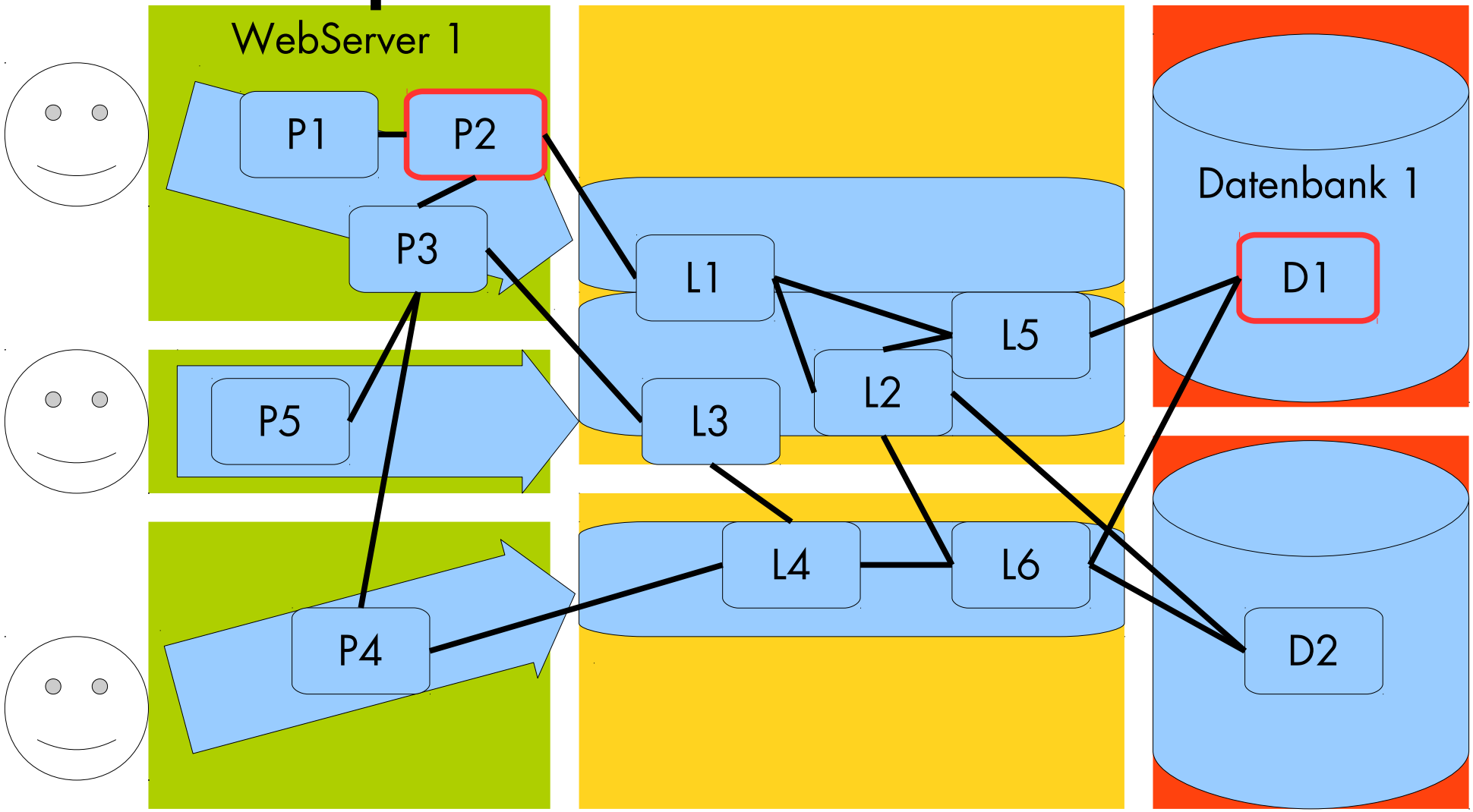
Transparenz damit realisierbar...



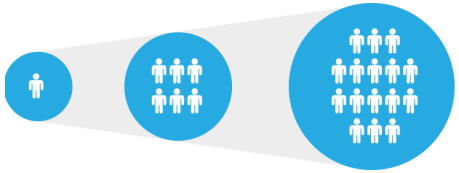
Zugriffstransparenz

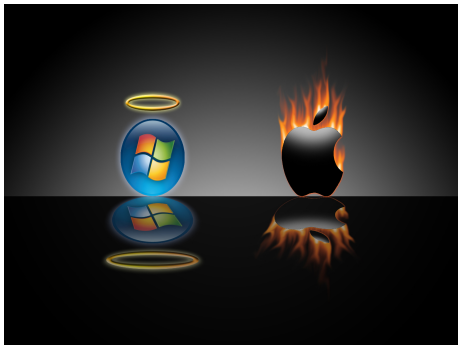


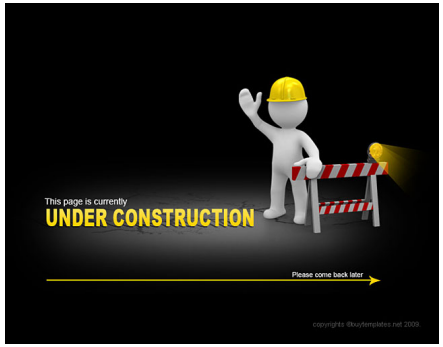
Ortstransparenz

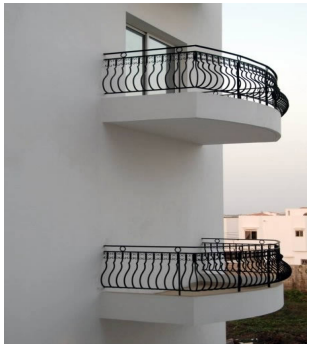


NFA damit realisierbar

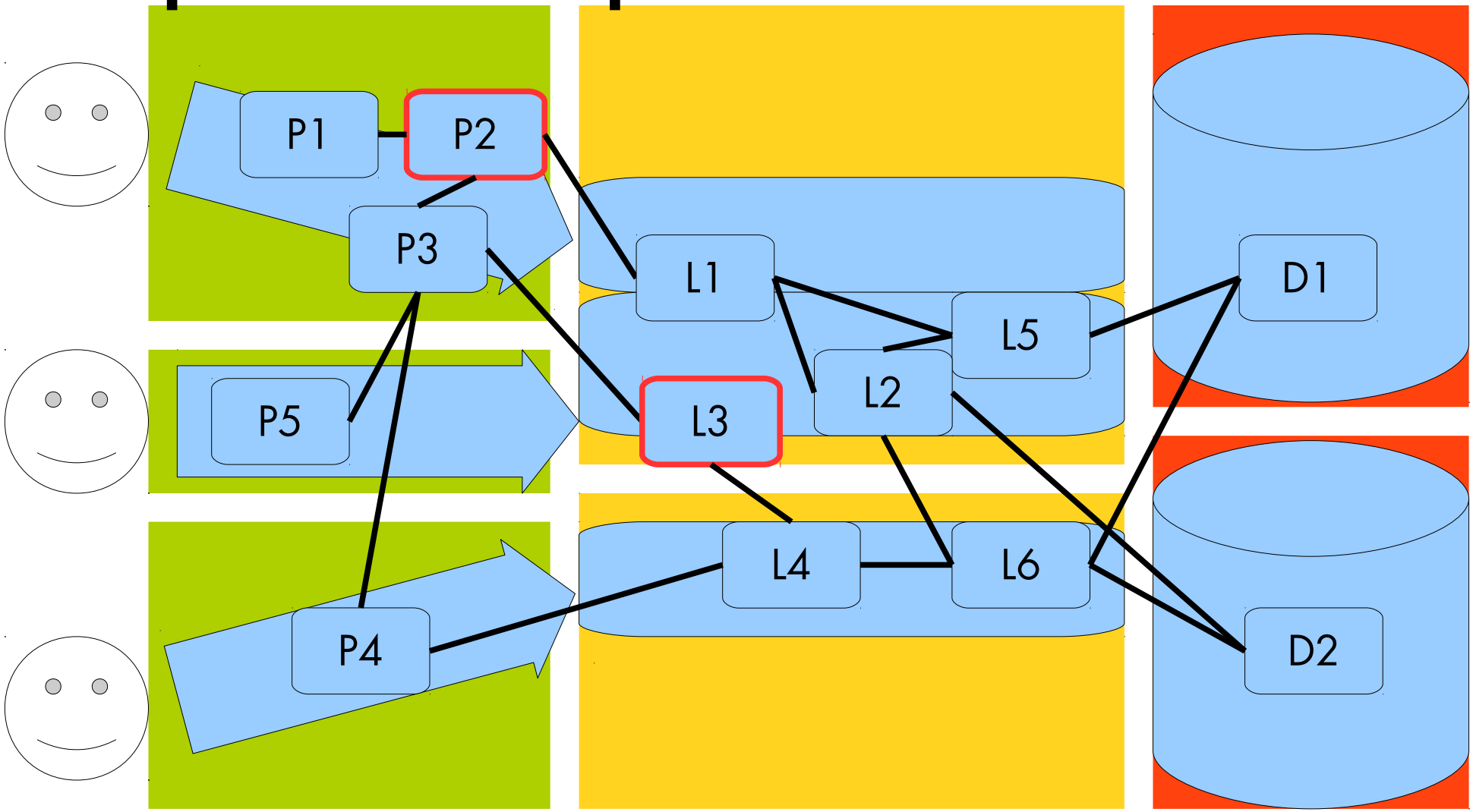




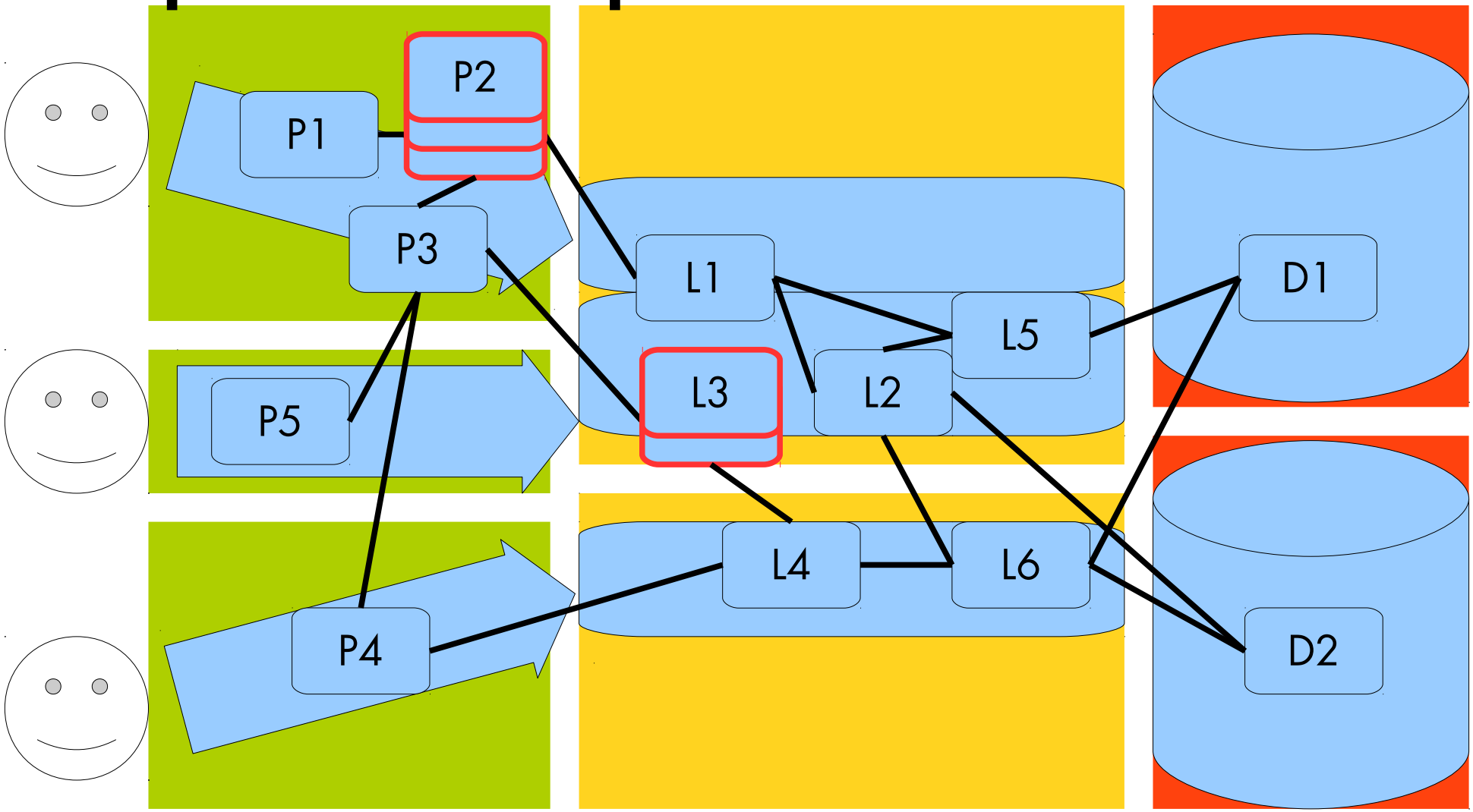




Replikationstransparenz



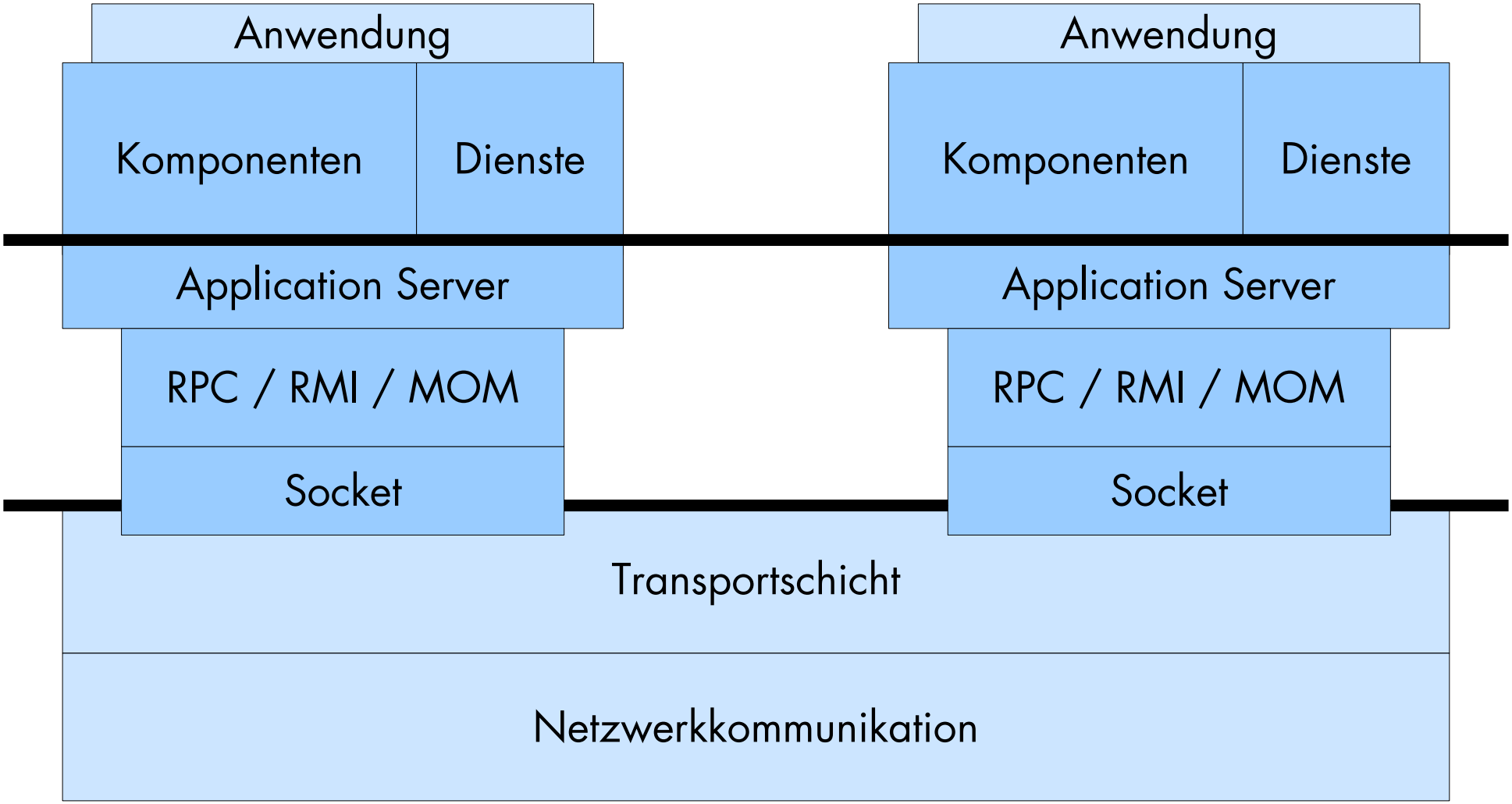
Replikationstransparenz

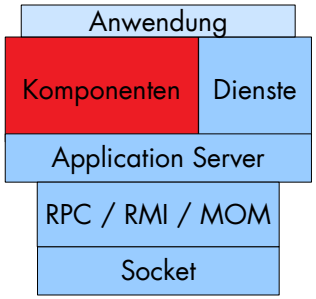


Anwendungsorientierte Middleware

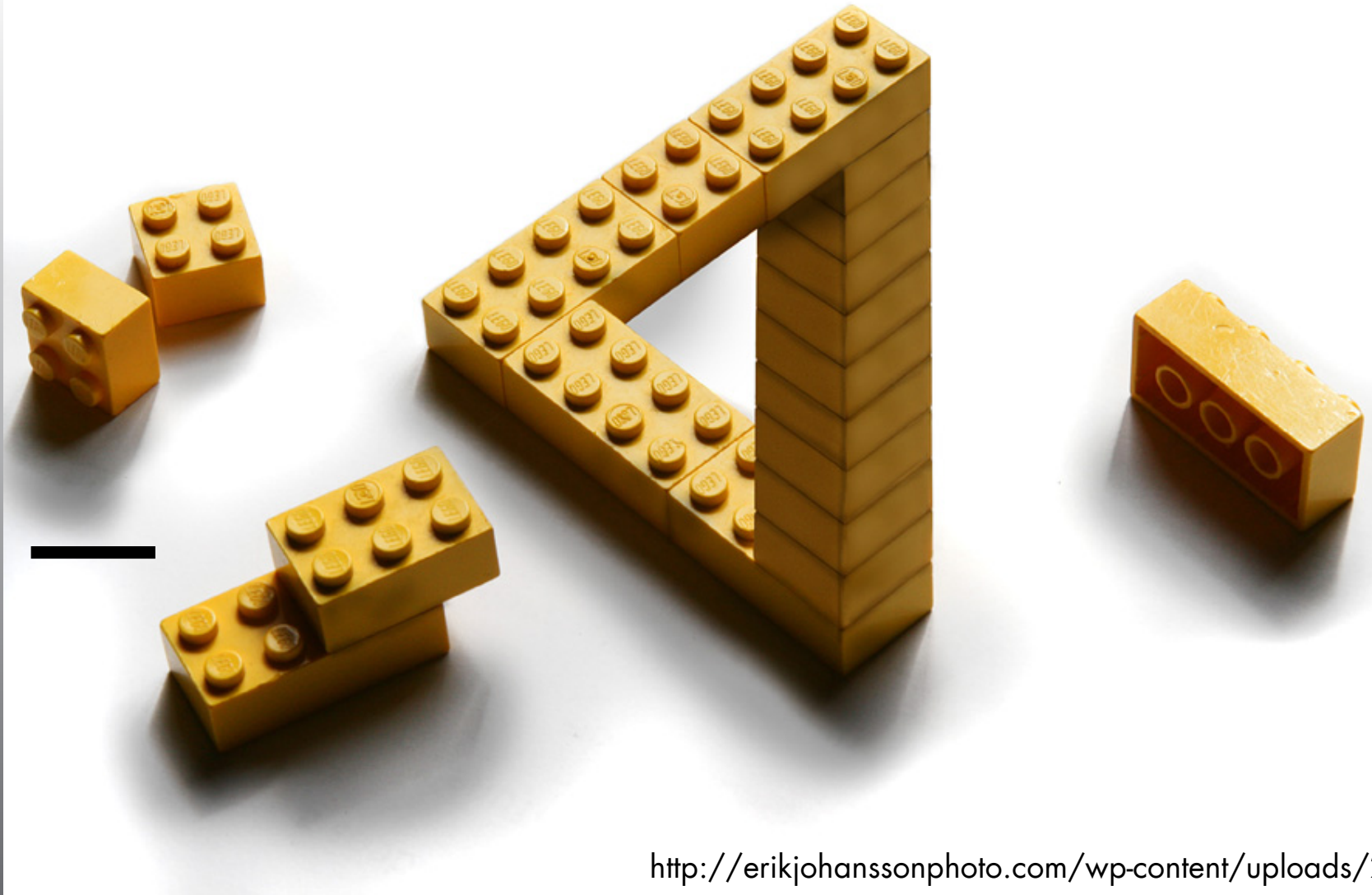
- Erweiterung der kommunikationsorientierten Middleware
  - Unterstützende Dienste
  - Eine Laufzeitumgebung für Komponenten
- Bestehend aus
  - Komponentenmodell
  - Dienstkomponenten
  - Laufzeitumgebung
- Durch Application-Server realisiert

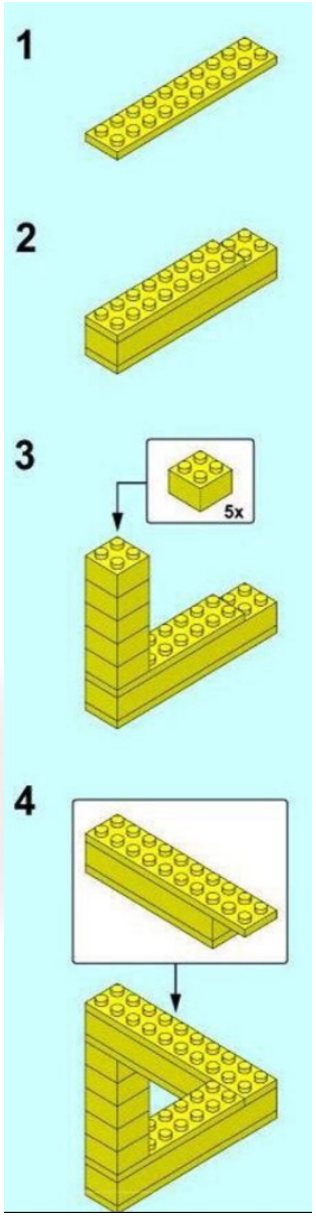
Anwendungsorientierte Middleware



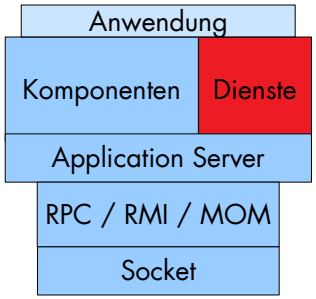


Componentware-Paradigma



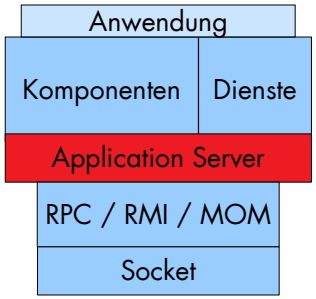


http://erikjohanssonphoto.com/wp-content/uploads/2012/05/lego_1.jpg



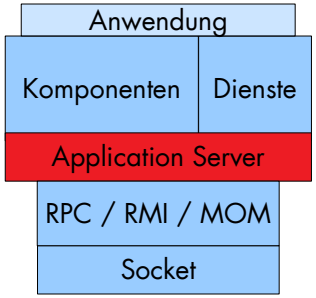
Dienste

- Middleware bietet für Anwendungen definierte Dienste an
  - Anwendungsunabhängig
  - Beliebig wiederverwendbar
  - Implizit über Laufzeitumgebung verfügbar
  - In der Regel technische Funktionen
- Schnittstellen in Spezifikation festgelegt
- Beispiele anwendungsorientierter Dienste
  - Namensdienst
  - Sitzungsverwaltung
  - Transaktionsverwaltung
  - Persistenzdienst



Application Server

- Stellt Laufzeitumgebung für Anwendungen zur Verfügung
- Bietet darüber hinaus zusätzliche Dienste für Anwendungen
- Verfügt idR. über ein Komponentenmodell für die Anwendungen
- Konzentration auf Unterstützung der Anwendungslogik
- Beispiel
  - IBM – Websphere
  - SAP NetWeaver
  - Jboss
  - Glassfish



Laufzeitumgebung

- Laufzeitumgebung des Betriebssystems nicht ausreichend
- Laufzeitumgebung der Middleware setzt darauf auf (Container)
- Stellt Funktionalität unabhängig von Hardware und Betriebssystem
- Realisiert damit komponentenorientierte Anwendungsentwicklung
- Aufgaben:
  - Ressourcenverwaltung: Hauptspeicher, Prozessorzeit
  - Nebenläufigkeit: Prozesse, Threads
  - Verbindungsverwaltung: Pooling (Verwaltung freier Verbindungen)
  - Verfügbarkeit: Fail-over, Load-balancing
  - Sicherheit: Authentisierung, Autorisierung, Integrität, Verschlüsselung

Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

1) Beispiel Namensdienst

- Dient der Veröffentlichung von Diensten und deren Auffinden
- Mapping von technischen Details der Dienste zu eindeutigen Namen
- Einträge enthalten Referenzen von Ressourcen
  - IP-Adresse / Port
  - ProzessIdentifikator / Objekt
- Dienstanbieter stellen Referenzen in den Namensdienst
- Client erhält Referenz aus Namensdienst über eindeutigen Namen
- Danach Aufbau der Verbindung zur Ressource
- Beispiele Standard-Schnittstellen für Namensdienste
  - Java Naming and Directory Interface (JNDI) in EJB
  - Interface Naming Service (INS) in CORBA

2) Beispiel Sitzungsverwaltung

- Sitzung als Kommunikationsverbindung zwischen Client und Server
- Relevante Daten werden für die Sitzung zwischengespeichert
- Unterscheidung nach
  - Transiente (flüchtige) Daten – Löschung nach Sitzungsende (Warenkorb)
  - Persistente Daten – dauerhafte Speicherung auf Datenträger (Bestellung)
- Mehrere Sitzungen durch entsprechenden Dienst verwaltet
  - Koordination der Kommunikationswege
  - Zuordnung zu zwischengespeicherten Daten für Sitzung
- Einsatz insbesondere bei interaktiven Anwendungen

3) Beispiel Persistenzdienst

- Wichtige Säule datenzentrierter verteilter Anwendungen
- Bietet Schnittstelle zu persistenten Speichermedien (Datenbank)
- Ziel ist transparente Persistierung relevanter Daten durch den Dienst
  - Führt Transformation für heterogene Medien durch
  - Abstrahiert von Schnittstellenspezifika der Medien
- Beispiel objektrelationaler Mapper (OR-Mapper)
  - Abbildung zwischen OO-Programmiersprache und relationaler DB
  - Persistierung der Objekte über Persistenzdienst
  - Verteilung der Attribute auf Tabellen und Felder transparent
  - Queries für heterogene DB-Systeme transparent

4) Beispiel Transaktionsverwaltung

- Zentraler Dienst interaktiver, datenzentrierter Anwendungen
- Wesentlich für Erhalt der Datenkonsistenz
- Datenkonsistenz als gültiger Zustand aller persistenter Daten
- Parallelzugriffe gefährden Datenkonsistenz
- Transaktionsmechanismus stellt Datenkonsistenz sicher

Transaktion

- Folge von Operationen, die logische, untrennbare Einheit bilden
- Sicherstellen der ACID-Eigenschaften
  - Atomarität (Atomicity): Alle oder keine Aktion ausgeführt
  - Konsistenz (Consistency): Konsistenter Zustand vorher und nachher
  - Isoliertheit (Isolation): Keine gegenseitigen Störungen von Transaktionen
  - Dauerhaftigkeit (Durability): Persistenter Endzustand nach Transaktion
- Steuerungsbestandteile
  - Beginn (engl. begin)
  - Bestätigung (engl. commit)
  - Zurücksetzen (engl. rollback)

Verteilte Transaktion 2-Phasen-*Commit*

- Konsistenzsicherung bei Zugriff auf Daten in verteilten Systemen
- Realisierung über 2-Phasen-*Commit*-Protokoll (2PC)
- Transaktionverwalter übernimmt verteilte Koordination
- Phase 1: Abstimmungsphase (engl. *Commit-Request-Phase*)
  - Lokale Ressourcenallokation aller Teilnehmer
  - Teilnehmer führen lokal Transaktionsschritte aus
  - Schreiben Ergebnis nicht selbstständig fest
  - Unterrichtet Transaktionsverwalter über Erfolg bzw. Fehlschlag

Verteilte Transaktion 2-Phasen-*Commit*

- Phase 2: Entscheidungsphase (engl. *Commit-Phase)*
  - Alle Beteiligten haben ihre Zustimmung/Ablehnung zurück gemeldet
  - Transaktionsverwalter informiert alle Beteiligten über
    - Den Vollzug der Transaktion (globales *Commit*)
    - Das Zurücksetzen (globales *Rollback*), da mindestens eine Ablehnung
  - Teilnehmer schreiben lokale Transaktion fest (lokales *Commit*) bzw. setzen die Änderungen zurück (lokales *Rollback*)

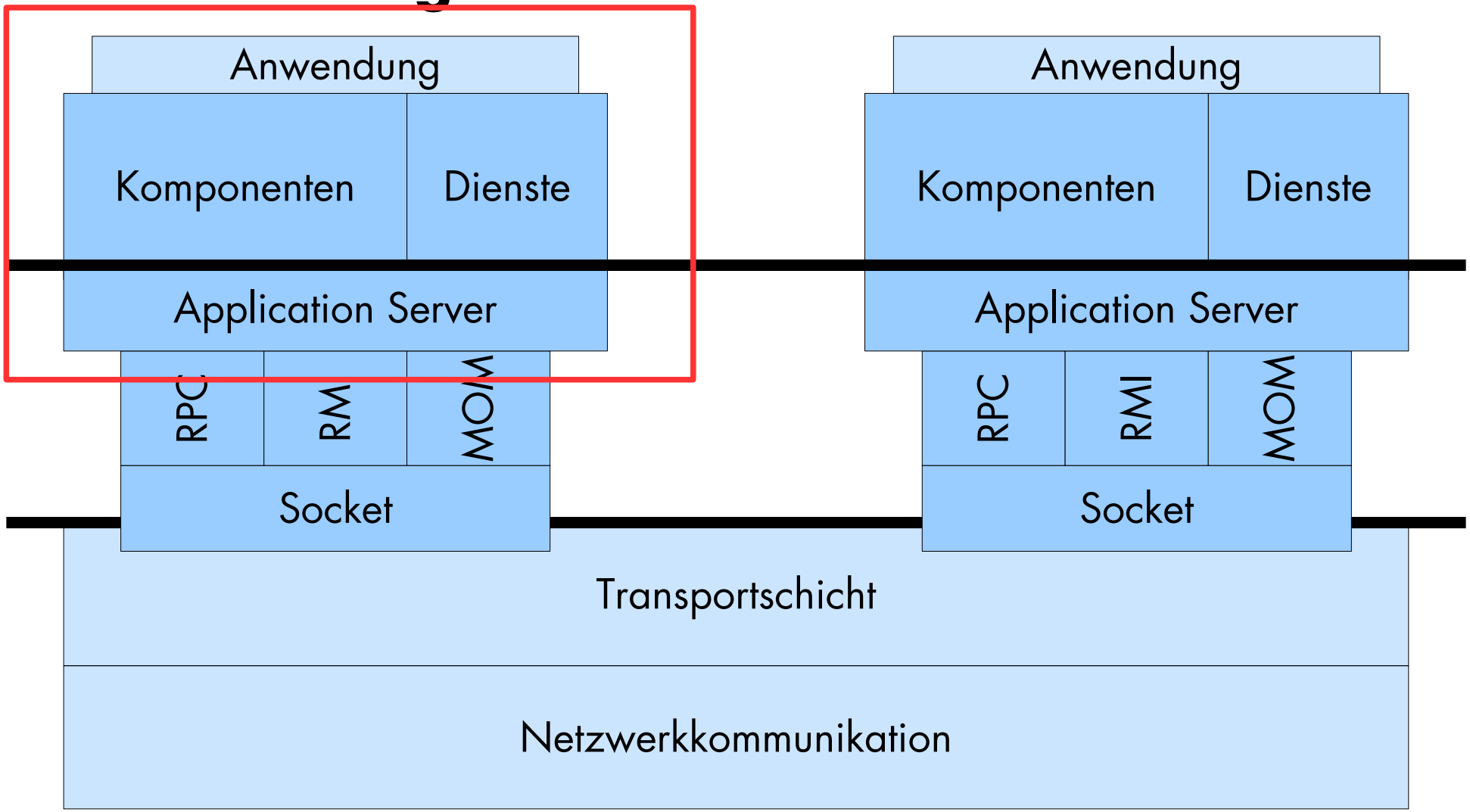
Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

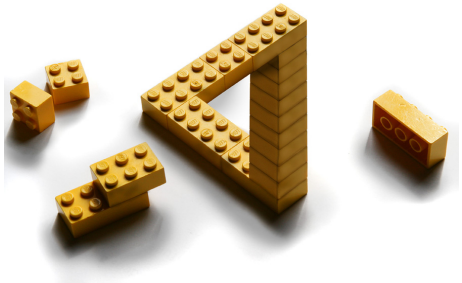
Komponentenmodell /-paradigma

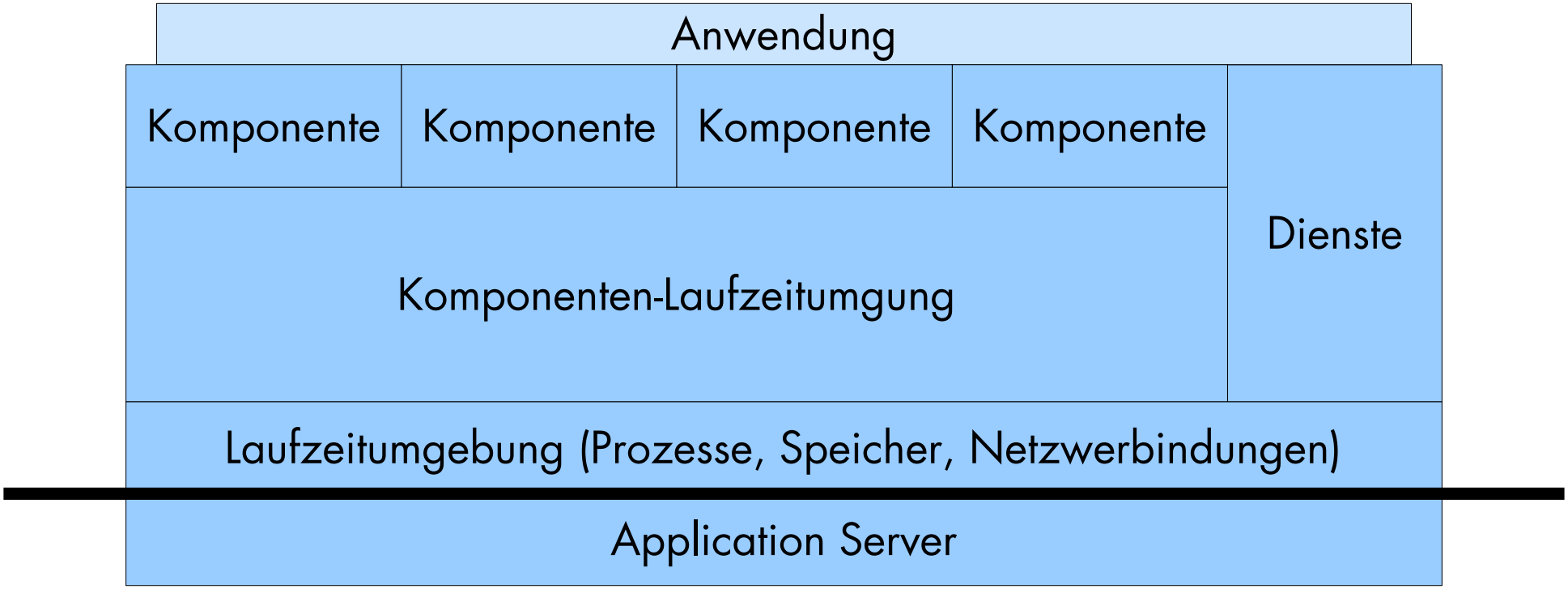
- Komponentenmodell
  - Konzeptionelle Basis komponentenbasierter verteilter Anwendungen
  - „Erweiterung“ des objektorientierten Paradigmas
- Komponenten als „große Objekte“
  - Dienen zur Strukturierung der Anwendung
  - Kapseln Teile der Anwendung
  - Bieten Funktionalitäten über Schnittstellen an
- Vorteile des Komponentenmodells
  - Strikte Trennung von fachlichen und technischen Aspekten
  - Realisierung des Baukastenprinzips für Entwicklungen
  - Vereinfachung von Entwicklung und Betrieb verteilter Anwendungen

Anwendungsorientierte Middleware



Komponentenmodell

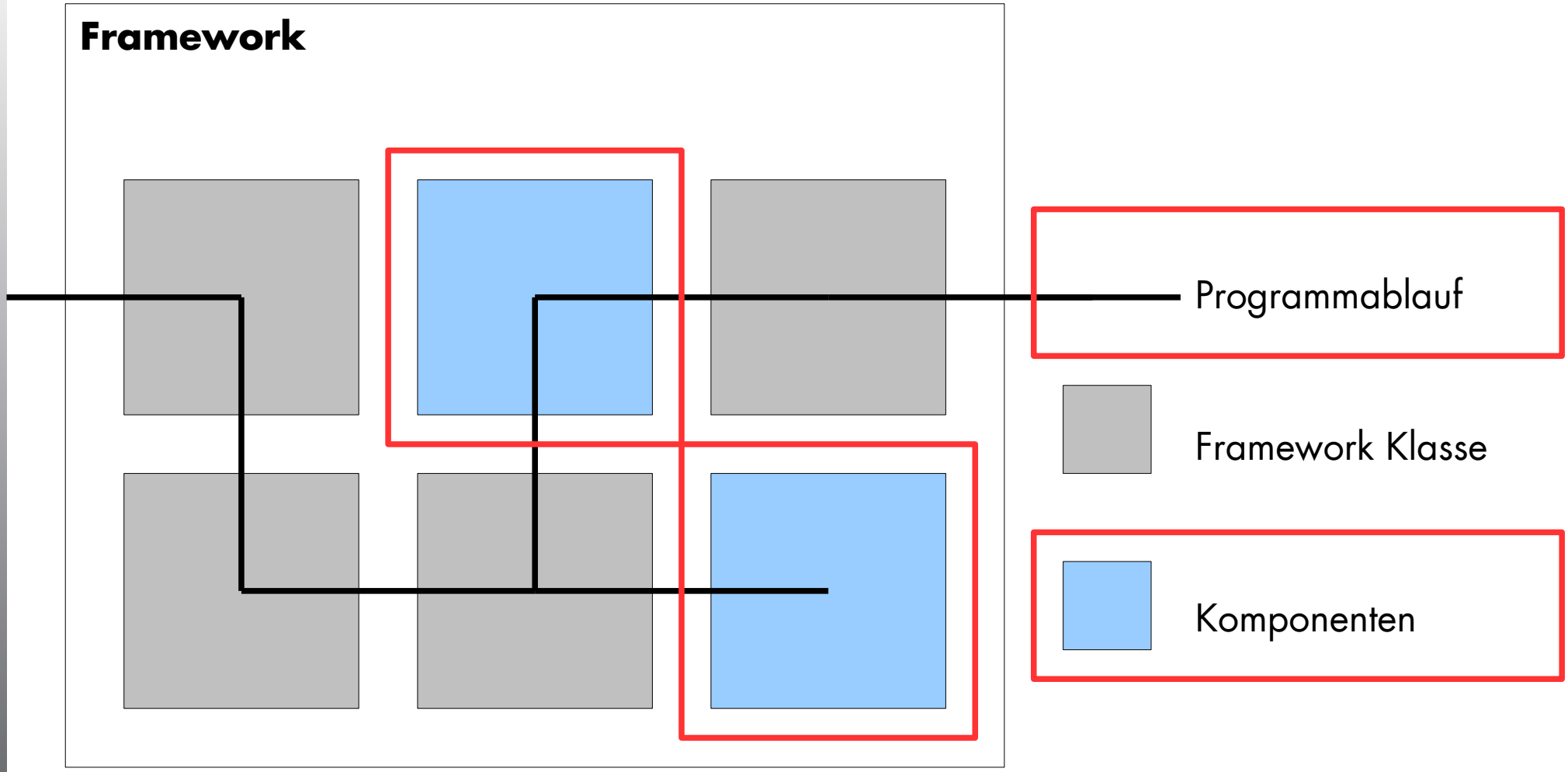




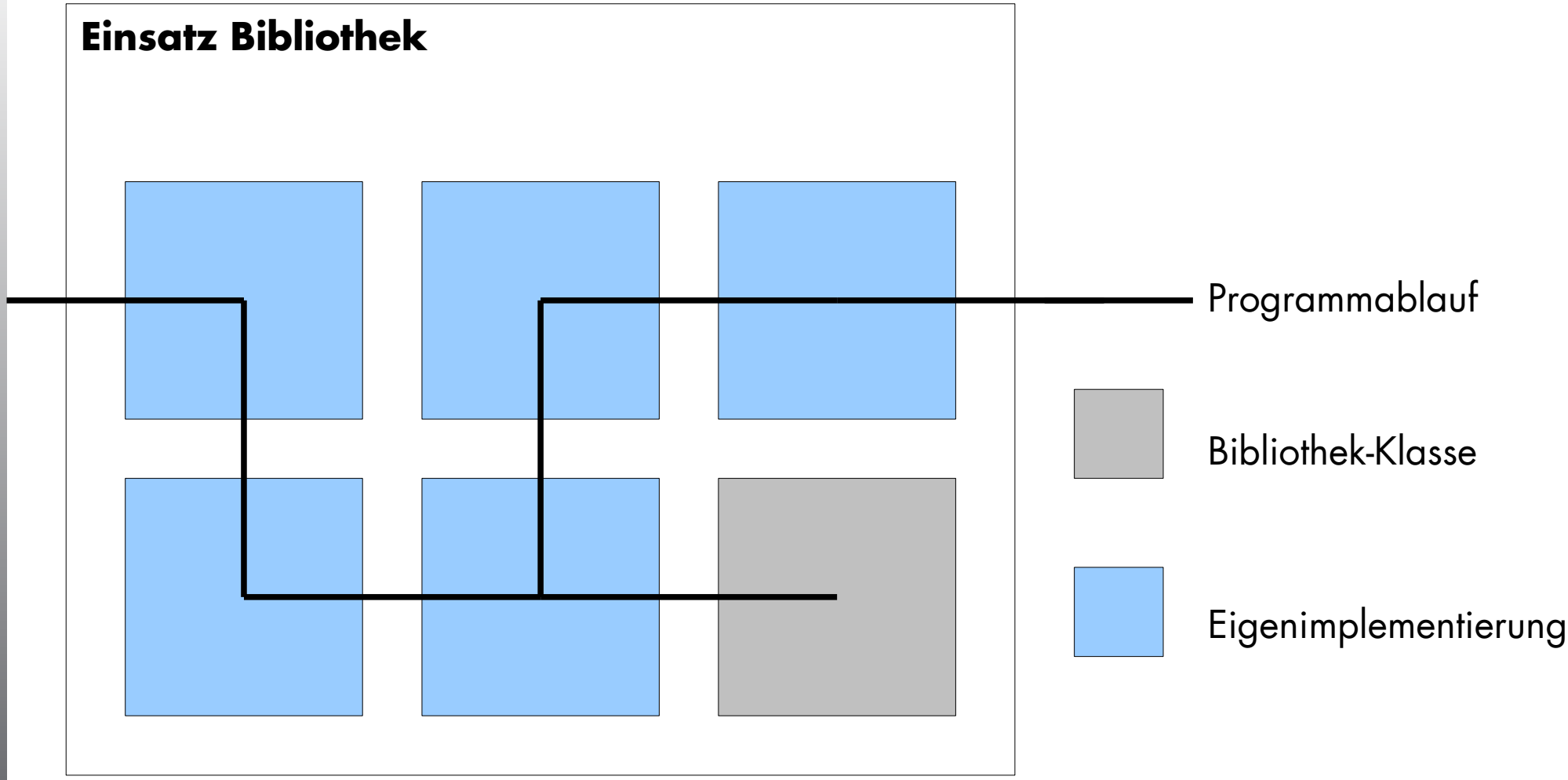
Elemente des Komponentenmodells

- Komponentenbegriff
  - Definiert durch das jeweilige Komponentenmodell
    - Definiert Art&Anzahl der Schnittstellen
    - Definiert Art&Anzahl der Komponenteneigenschaften
  - Legt Entwicklungsstruktur für Komponenten fest
  - Legt Integration in Komponentenlaufzeitumgebung fest
- Komponente
  - Definition einer verbindlichen Schnittstelle
  - Explizite Deklaration externer Abhängigkeiten
  - Unabhängige Ausführung bei Erfüllung aller Abhängigkeiten
  - Einfache Wiederverwendung durch Komposition

Komponenten-Framework



Unterschied zur Bibliothek



Elemente des Komponentenmodells

- Schnittstellenverträge
  - Bestimmen Abläufe zwischen den Komponenten
  - Bestimmen Abläufe zwischen Komponenten und Laufzeitumgebung
  - Definition der Nutzungsform der Dienste
- Komponentenlaufzeitumgebung
  - Realisiert Nutzung von Komponenten zur Erstellung von Anwendungen
  - Eigene Laufzeitumgebung für Komponenten (Container)
  - Verwaltet den Komponentenlebenszyklus
  - Realisiert die Nutzung der Dienste durch die Komponenten

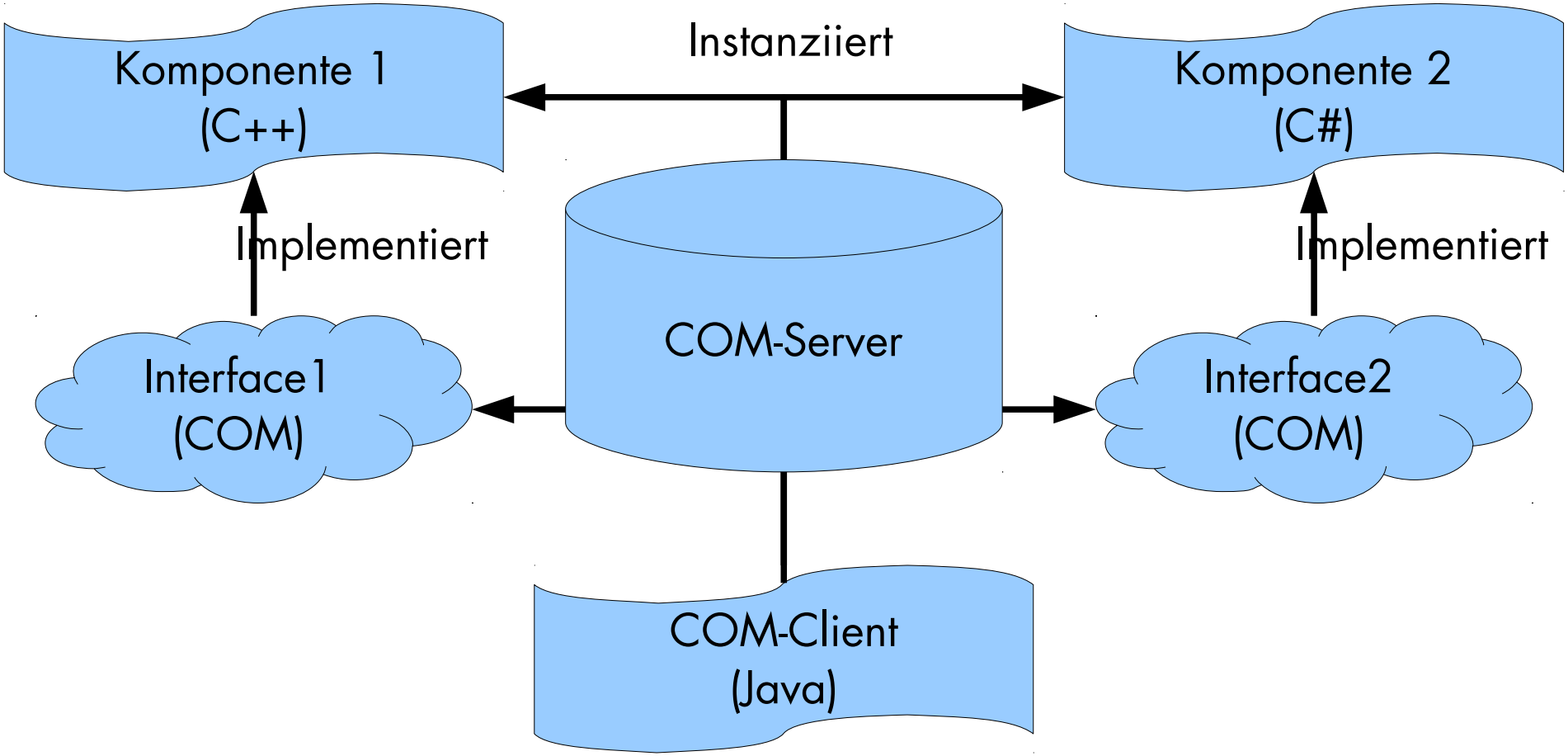
Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

Component Object Model (COM)

- Interaktionsmechanismus für lokale Softwarekomponenten (1993)
- Interprozesskommunikation und dynamische Objekterzeugung
- COM basiert auf Client/Server-Prinzip
- Auch über (Windows-)Betriebssystemgrenzen hinweg (DCOM)
- COM-Komponenten als DLLs oder ausführbares Programme (*.exe)
- Leichte Wiederverwendung von Programmcode
- COM-Komponenten sind
  - Sprachunabhängig
  - Versionsunabhängig
  - Plattformunabhängig

Arbeiten mit (D)COM



Backup

Bestandteile

- COM-Komponente
  - Aufrufbaren Funktionen über ein oder mehrere COM-Interfaces
- COM-Interface
  - Dient der Kommunikation vom Client zum Server
  - „Instantiierung“ als ProxyObjekt mit Eindeutige Identifikationsnummer
  - Pro COM-Komponente mehrere Interfaces möglich
- COM-Client
  - Nutzung der Funktionaliät von COM-Komponenten
  - Kennt die Funktionen der COM-Komponente nur über das COM-Interface
- COM-Server
  - Erstellt COM-Komponenten und bietet sie an
  - Unterscheidung In-Process- /Local- /Remote-Server

Interface Beispiel

```
//
// Interface mathematik.h
//
class IStandardMathFunctions  :  public IUnknown
{
public:
    STDMETHOD(Addieren)(long, long, long*)
    STDMETHOD(Subtrahieren)(long, long, long*)
};

class IAdvancedMathFunctions  :  public IUnknown
{
public:
    STDMETHOD(Fibonacci)(short, long*)
}
```

https://de.wikipedia.org/wiki/Component_Object_Model

Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA

Common Object Request Broker Architecture

- Entwickelt von der Object Management Group (OMG)
- Spezifikation für eine objektorientierte Middleware
- Kern bildet ein Object Request Broker (ORB)
- Stellt plattformübergreifende Protokolle und Dienste zur Verfügung
- Eine Spezifikation, mehrere herstellerspezifische Implementierung
- Überwindung Heterogenität von Programmiersprache und System
- Analog JVM, die Systemheterogenität überwindet

Interface Definition Language (IDL)

- Formale Schnittstellenbeschreibung für Daten und Signaturen
- Beschreibt angebotene Funktionalität des Servers
- Programmiersprachenunabhängig

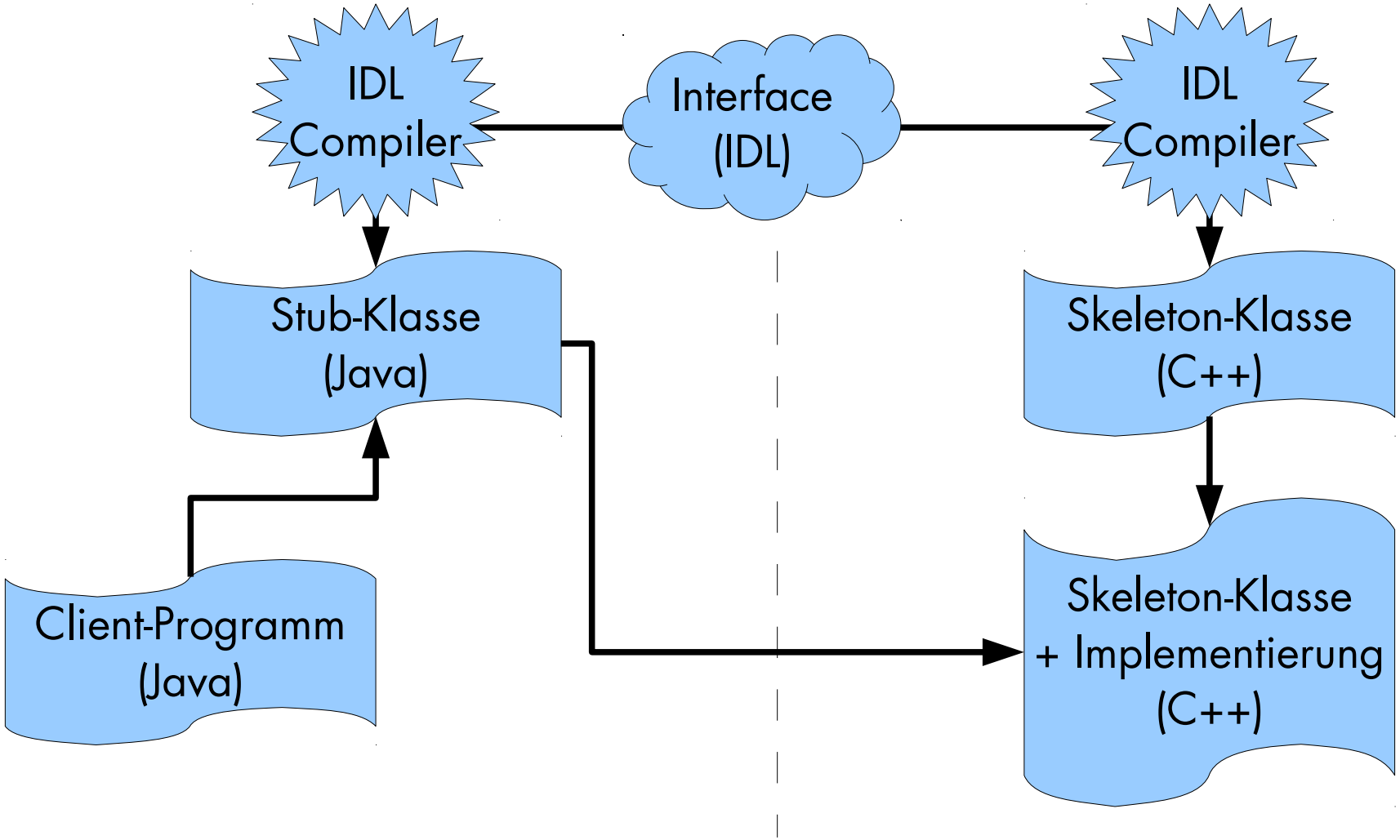
```
module finance {
  interface account {
    // The account owner and balance.
    readonly attribute string owner;
    readonly attribute float balance;
    // Operations available on the account.
    void makeLodgement(in float amount, out float newBalance);
    void makeWithdrawal(in float amount, out float newBalance);
  };
};
```

https://de.wikipedia.org/wiki/Component_Object_

Interface Definition Language (IDL)

- IDL-Compiler übersetzt in Objektmodell der jeweiligen Sprache
- IDL-Compiler ist Bestandteil des herstellerspezifischen ORB
- Generiert den Quelltext für Stubs und Skeletons (ähnlich RMI)
- Transparenz der Komplexität der Netzwerkschnittstelle
- Transparenz des Methodenaufrufs

Arbeiten mit CORBA



Entwicklung mit CORBA

- 1) Definition der Schnittstelle per IDL
- 2) Generierung der Skeleton-Klassen per IDL-Compiler
- 3) Implementieren der Logik auf Serverseite
- 4) Generierung der Stub-Klassen per IDL-Compiler
- 5) Nutzung der Stub-Klassen für verteilte Funktionalität
- Programmiersprachen unabhängig
- Interprozesskommunikation für Client und Server transparent
- Arbeitsaufwand in der Client-Server-Entwicklung stark reduziert

CORBA-Dienste

- Naming Service – Objekte nach Namen identifizieren
- Trading Service – Objekte nach Eigenschaften identifizieren
- Life Cycle Service – Objekte Kopieren, Verschieben und Löschen
- Weitere Services
  - Event Service
  - Relationship Service
  - Persistent Object Service
  - Concurrency Control Service
  - Transaction Service
  - Property Service
  - Licensing Service
  - …

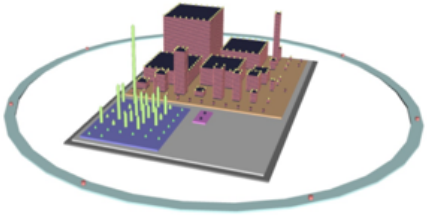
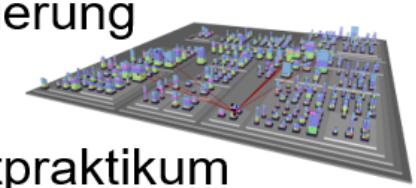
Übersicht

- Kommunikationsorientierte Middleware
  - Prozedurales Modell
  - Objektorientiertes Modell
  - Nachrichtenbasiertes Modell
- Anwendungsorientierte Middleware
  - Dienste
  - Komponentenmodell
  - Component Object Model (COM)
  - CORBA
  - JEE…

Vielen Dank für Ihre Aufmerksamkeit!

GISA-WERKSSTUDENT (M/W) IN HALLE/LEIPZIG

SOFTWARE-VISUALISIERUNG VON SAP/ABAP



- Ihre Aufgaben und Ziele
  - Unterstützung bei der Visualisierungen von SAP/ABAP
  - Integration von Softwarevisualisierung in den Projektbetrieb
- Ihre Qualifikationen und Kompetenzen
  - Interesse und gute Kenntnisse der Softwareentwicklung
  - Interesse an Forschung zur Softwarevisualisierung
- Ihre Perspektiven
  - Vergüteter Werksstudent der GISA als Pflichtpraktikum
  - Parallel Möglichkeit zur Bearbeitung der Bachelorarbeit
  - Aktive Betreuung im besten Team der GISA